

**TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**  
**KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN HỆ CƠ SỞ DỮ LIỆU**

**Tài liệu dịch chương 11**

*Giảng viên hướng dẫn:*      Đào Thị Ngọc Hân

*STT nhóm:* 11 ( lớp 67IT4 )

*Sinh viên thực hiện:*      Hoàng Văn Đức ( nhóm trưởng )

Nguyễn Quang Nghĩa

Nguyễn Văn Kiên

Nguyễn Mạnh Cường

*Hà Nội, 10-2023*

# Chương 11: Database Performance Tuning and Query Optimization ( Tối ưu hiệu suất cơ sở dữ liệu và tinh chỉnh truy vấn)

Sau khi hoàn thành chương này, bạn sẽ có khả năng:

1. Nhận biết các thủ tục liên quan đến tinh chỉnh hiệu suất cơ sở dữ liệu.
2. Mô tả cách hệ quản trị cơ sở dữ liệu (DBMS) xử lý các truy vấn SQL trong ba giai đoạn chính của nó.
3. Giải thích vai trò của các chỉ mục trong việc tăng tốc truy cập dữ liệu.
4. Phân biệt giữa bộ tối ưu hóa dựa trên quy tắc ( rule-based optimizer ) và bộ tối ưu hóa dựa trên chi phí (cost-based optimizer).
5. Mô tả một số thực hành phổ biến được sử dụng để viết mã SQL hiệu quả.
6. Giải thích cách đặt ra các truy vấn và tinh chỉnh DBMS để đạt được hiệu suất tối ưu.

**PREVIEW:** Tinh chỉnh hiệu suất cơ sở dữ liệu ( database performance tuning) là một chủ đề quan trọng, tuy nhiên, thường ít được bàn về trong chương trình học về cơ sở dữ liệu. Hầu hết các cơ sở dữ liệu được sử dụng trong lớp học thường chỉ có một số bản ghi trên mỗi bảng. Kết quả là, trọng tâm thường là làm cho các truy vấn SQL hoạt động theo cách dự kiến mà không xem xét tính hiệu quả của quá trình truy vấn. Trên thực tế, ngay cả môi trường truy vấn hiệu quả nhất cũng không tạo ra sự cải thiện hiệu suất rõ ràng so với môi trường truy vấn hiệu quả nhất khi chỉ có 20 hoặc 30 hàng (records) trong bảng được truy vấn. Thật không may, sự thiếu chú ý đó đối với hiệu suất truy vấn có thể dẫn đến kết quả chậm không chấp nhận trong thế giới thực khi truy vấn được thực hiện trên hàng chục triệu bản ghi. Trong chương này, bạn sẽ tìm hiểu những gì cần thiết để tạo ra một môi trường truy vấn hiệu quả hơn.

| Data Files and Available Formats |           |        |        |        |
|----------------------------------|-----------|--------|--------|--------|
|                                  | MS Access | Oracle | MS SQL | My SQL |
| CH11_SaleCo                      | ✓         | ✓      | ✓      | ✓      |

Data Files Available on [cengagebrain.com](http://cengagebrain.com)

**NOTE** Bởi vì cuốn sách này tập trung vào cơ sở dữ liệu, chương này chỉ trình bày các yếu tố ảnh hưởng trực tiếp đến hiệu suất cơ sở dữ liệu. Ngoài ra, vì các kỹ thuật điều chỉnh hiệu suất có thể cụ thể cho từng hệ quản trị cơ sở dữ liệu (DBMS), nội dung trong chương này có thể không áp dụng trong mọi trường hợp và không nhất thiết phù hợp với tất cả các loại DBMS. Chương này được thiết kế để xây dựng nền tảng cho sự hiểu biết tổng quan về các vấn đề điều chỉnh hiệu suất cơ sở dữ liệu và giúp bạn lựa chọn các chiến lược điều chỉnh hiệu suất phù hợp. (Để biết thông tin cập nhật nhất về cách điều chỉnh cơ sở dữ liệu của bạn, hãy tham khảo tài liệu của nhà cung cấp cơ sở dữ liệu.)

## 11-1 Database Performance-Tuning Concepts

Một trong các chức năng chính của hệ thống cơ sở dữ liệu là cung cấp câu trả lời kịp thời cho người dùng cuối. Người dùng cuối tương tác với Hệ quản trị cơ sở dữ liệu (DBMS) thông qua việc sử dụng các truy vấn để tạo thông tin, sử dụng chuỗi sau đây:

1. Ứng dụng cuối người dùng (client-end) tạo ra một truy vấn.
2. Truy vấn được gửi đến Hệ quản trị cơ sở dữ liệu (DBMS – server end).
3. Hệ quản trị cơ sở dữ liệu (DBMS – server end) thực thi truy vấn.

**Tinh chỉnh hiệu suất cơ sở dữ liệu** là một tập hợp các hoạt động và quy trình được thiết kế để giảm thời gian phản hồi của hệ thống cơ sở dữ liệu, đảm bảo rằng một truy vấn của người dùng cuối được xử lý bởi bởi hệ quản trị cơ sở dữ liệu (DBMS) trong thời gian tối thiểu.

#### 4. Hệ quản trị cơ sở dữ liệu gửi tập dữ liệu

kết quả đến ứng dụng cuối người dùng (client-end).

Người dùng cuối mong đợi các truy vấn của họ trả kết quả càng nhanh càng tốt. Làm thế nào bạn biết rằng hiệu suất của một cơ sở dữ liệu là tốt? Hiệu suất cơ sở dữ liệu tốt khá khó đánh giá. Làm thế nào bạn biết xem thời gian phản hồi của một truy vấn là 1,06 giây có đủ tốt không? Vì vậy, hiệu suất cơ sở dữ liệu cần được theo dõi chặt chẽ và điều chỉnh đều đặn. **Tinh chỉnh hiệu suất cơ sở dữ liệu** đề cập đến một tập hợp các hoạt động và thủ tục được thiết kế để giảm thời gian phản hồi của hệ thống cơ sở dữ liệu, đảm bảo rằng truy vấn của người dùng cuối được xử lý bởi DBMS trong thời gian tối thiểu.

Thời gian cần cho một truy vấn trả về tập dữ liệu kết quả phụ thuộc vào nhiều yếu tố, có xu hướng phổ quát (wide-ranging) và thay đổi giữa các môi trường (environments) và giữa các nhà cung cấp (vendors). Nói chung, hiệu suất của một DBMS điển hình bị ràng buộc bởi ba yếu tố chính: sức mạnh xử lý CPU, bộ nhớ chính có sẵn (RAM), và công suất nhập/xuất (ổ cứng-hard disk và mạng-network). Bảng 11.1 liệt kê một số thành phần hệ thống và tóm tắt hướng dẫn chung để đạt được hiệu suất truy vấn tốt hơn.

Tất nhiên, hệ thống sẽ hoạt động tốt nhất khi tài nguyên phần cứng (hardware) và phần mềm (software) của nó được tối ưu hóa. Tuy nhiên, trong thế giới thực, tài nguyên không giới hạn không phải là điều thông thường; luôn tồn tại các ràng buộc nội và ngoại. Do đó, các thành phần hệ thống nên được tối ưu hóa để đạt được công suất tốt nhất có thể với tài nguyên hiện có (và thường là giới hạn), đó là lý do tại sao việc điều chỉnh hiệu suất cơ sở dữ liệu quan trọng.

Tinh chỉnh hiệu suất của một hệ thống đòi hỏi một cách tiếp cận toàn diện. Nghĩa là, tất cả các yếu tố phải được kiểm tra để đảm bảo mỗi yếu tố hoạt động ở mức tối ưu và có đủ tài nguyên để giảm thiểu sự cố đường ống. Bởi vì thiết kế cơ sở dữ liệu là yếu tố quan trọng quyết định hiệu suất của hệ thống cơ sở dữ liệu, nên quan trọng lặp lại điều nguyên tắc trong cuốn sách này:

Hiệu suất cơ sở dữ liệu tốt bắt đầu từ thiết kế cơ sở dữ liệu tốt (**Good database performance starts with good database design**). Không có mức tinh chỉnh nào có thể khiến cho một cơ sở dữ liệu thiết kế kém hiệu suất bằng cách thiết kế cơ sở dữ liệu tốt.

TABLE 11.1

## HƯỚNG DẪN CHUNG ĐỂ CẢI THIỆN HIỆU SUẤT HỆ THỐNG

|          | SYSTEM<br>RESOURCES | CLIENT                                                                                                           | SERVER                                                                                                                                                                                                                                                                                                                          |
|----------|---------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hardware | CPU                 | CPU hai nhân hoặc cao cấp nhất có thể làm điều này.<br>"Công nghệ máy tính cá nhân ảo cũng có thể được sử dụng." | Công nghệ nhiều bộ xử lý nhất có thể (4 lõi trở lên)<br>Cụm máy tính mạng(sử dụng công nghệ tức thời hoặc cao cấp hơn)<br>"công nghệ máy chủ ảo cũng có thể được sử dụng"                                                                                                                                                       |
|          | RAM                 | Tối đa để tránh việc trao đổi bộ nhớ với đĩa của hệ điều hành.                                                   | Cực độ để tránh việc trao đổi bộ nhớ với đĩa của hệ điều hành.                                                                                                                                                                                                                                                                  |
|          | Storage             | Ổ cứng SATA/EIDE nhanh với đủ không gian trống đủ lớn<br><br>Ổ đĩa thể rắn (SSDs) để đạt được tốc độ nhanh hơn   | Nhiều ổ cứng tốc độ cao, dung lượng lớn<br><br>Giao diện ổ cứng nhanh (SAS / SCSI / Firewire / Fibre Channel)<br><br>Cấu hình RAID tối ưu hóa cho lưu lượng thông qua<br><br>Ổ đĩa thể rắn (SSDs) để đạt tốc độ nhanh hơn<br><br>Ổ cứng riêng cho hệ điều hành, hệ quản lý cơ sở dữ liệu (DBMS), và không gian lưu trữ dữ liệu. |
|          | Network             | Tốc độ kết nối cao                                                                                               | Tốc độ kết nối cao                                                                                                                                                                                                                                                                                                              |

|          |                       |                                                                                                                                    |                                                                                                                                 |
|----------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Software | Operating system (OS) | Hệ điều hành 64-bit để có không gian địa chỉ lớn hơn<br><br>Được điều chỉnh tinh chỉnh cho hiệu suất ứng dụng người dùng tốt nhất. | Hệ điều hành 64-bit để có không gian địa chỉ lớn hơn<br><br>Được điều chỉnh tinh chỉnh cho hiệu suất ứng dụng máy chủ tốt nhất. |
|          | Network               | Được điều chỉnh tinh chỉnh cho lưu lượng thông qua tốt nhất.                                                                       | Được điều chỉnh tinh chỉnh cho lưu lượng thông qua tốt nhất.                                                                    |
|          | Application           | Tối ưu hóa SQL trong ứng dụng khách hàng.                                                                                          | Tối ưu hóa máy chủ quản lý cơ sở dữ liệu (DBMS) để đạt hiệu suất tốt nhất.                                                      |

Điều này đặc biệt đúng khi tái thiết kế cơ sở dữ liệu hiện có, trong trường hợp người dùng cuối kỳ vọng có được sự tăng cường hiệu suất không thực tế từ các cơ sở dữ liệu cũ.

Điều gì tạo nên một thiết kế cơ sở dữ liệu tốt, hiệu quả? Từ góc độ tinh chỉnh hiệu suất, người thiết kế cơ sở dữ liệu cần đảm bảo rằng thiết kế sử dụng các tính năng trong hệ quản trị cơ sở dữ liệu (DBMS) để đảm bảo tính toàn vẹn và hiệu suất tối ưu của cơ sở dữ liệu. Chương này cung cấp kiến thức cơ bản giúp bạn tối ưu hóa hiệu suất cơ sở dữ liệu bằng cách chọn cấu hình máy chủ cơ sở dữ liệu phù hợp, sử dụng chỉ mục, hiệu cách tổ chức lưu trữ bảng và vị trí dữ liệu, và triển khai cú pháp truy vấn SQL hiệu quả nhất.

## 11-1a Performance Tuning: Client and Server

Nói chung, hoạt động tinh chỉnh hiệu suất cơ sở dữ liệu có thể chia thành hai phần: phần bên phía máy khách và phần bên phía máy chủ.

- Bên phía máy khách, mục tiêu là tạo ra một truy vấn SQL trả về kết quả chính xác trong thời gian ngắn nhất, sử dụng ít tài nguyên nhất tại máy chủ. Các hoạt động cần thiết để đạt được mục tiêu này thường được gọi là **tinh chỉnh hiệu suất SQL (SQL performance tuning)**.
- Bên phía máy chủ, môi trường của hệ quản trị cơ sở dữ liệu (DBMS) phải được cấu hình đúng cách để đáp ứng yêu cầu của máy khách một cách nhanh nhất, đồng thời

tối ưu hóa việc sử dụng tài nguyên hiện có. Các hoạt động cần thiết để đạt được mục tiêu này thường được gọi là **tinh chỉnh hiệu suất DBMS (DBMS performance tuning)**.

Hãy lưu ý rằng triển khai hệ quản trị cơ sở dữ liệu (DBMS) thường phức tạp hơn chỉ là cấu hình hai lớp máy khách/máy chủ. Phần mạng đóng một vai trò quan trọng trong việc truyền tải các thông điệp giữa máy khách và máy chủ; điều này đặc biệt quan trọng trong cơ sở dữ liệu phân tán. Tuy nhiên, trong chương này, chúng tôi giả định rằng mạng đã được tối ưu hóa hoàn toàn, và do đó, sự tập trung của chúng tôi nằm ở các thành phần của cơ sở dữ liệu. Ngay cả trong môi trường máy chủ/máy khách nhiều lớp bao gồm giao diện máy khách, phần mềm trung gian ứng dụng và máy chủ cơ sở dữ liệu, các hoạt động tinh chỉnh hiệu suất thường được chia thành các phần nhỏ để đảm bảo thời gian phản hồi nhanh nhất có thể giữa hai điểm thành phần bất kỳ. Người quản trị cơ sở dữ liệu phải làm việc chặt chẽ với nhóm mạng để đảm bảo lưu lượng cơ sở dữ liệu luôn mạch lạc trên cơ sở hạ tầng mạng. Điều này trở nên quan trọng hơn khi bạn xem xét rằng hầu hết các hệ thống cơ sở dữ liệu phục vụ cho người dùng phân tán về địa lý.

Chương này bao gồm các thực hành tinh chỉnh hiệu suất SQL trên phía máy khách và các thực hành tinh chỉnh hiệu suất DBMS trên phía máy chủ. Tuy nhiên, trước khi bạn bắt đầu tìm hiểu về quá trình tinh chỉnh, bạn phải trước hết nắm vững về các thành phần kiến trúc của DBMS và quy trình, cũng như cách các quy trình đó tương tác để đáp ứng yêu cầu của người dùng cuối.

## 11-1b DBMS Architecture

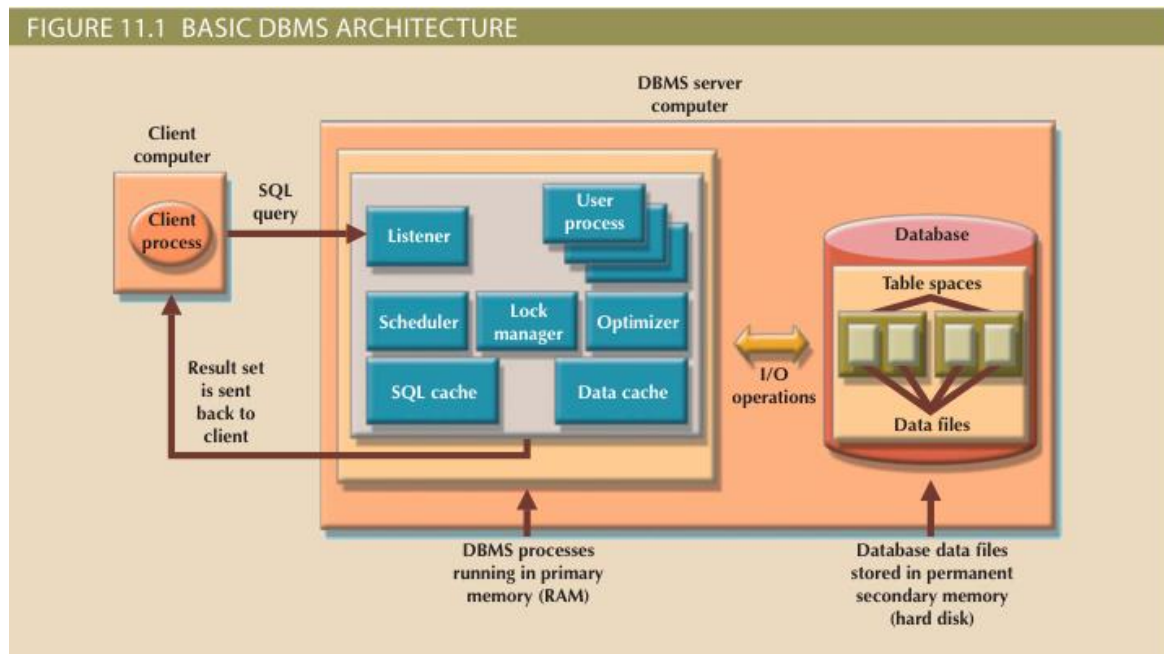
Kiến trúc của một hệ quản trị cơ sở dữ liệu (DBMS) được đại diện bởi các quy trình và cấu trúc (trong bộ nhớ và lưu trữ cố định) được sử dụng để quản lý một cơ sở dữ liệu. Các quy trình này hợp tác với nhau để thực hiện các chức năng cụ thể. Hình 11.1 minh họa kiến trúc cơ bản của DBMS.

Thành phần và chức năng theo hình 11.1:

- Tất cả dữ liệu trong một cơ sở dữ liệu được lưu trữ trong các tệp dữ liệu (**data file**). Một cơ sở dữ liệu doanh nghiệp điển hình thường bao gồm nhiều tệp dữ liệu. Một tệp dữ liệu có thể chứa các hàng từ một bảng duy nhất, hoặc nó có thể chứa các hàng từ nhiều bảng khác nhau. Người quản trị cơ sở dữ liệu (DBA) xác định kích thước ban đầu của các tệp dữ liệu tạo nên cơ sở dữ liệu; tuy nhiên, các tệp dữ liệu có thể

tự động mở rộng khi cần trong các đơn vị tăng trước được định trước gọi là **extents**. Ví dụ, nếu cần thêm không gian, DBA có thể xác định rằng mỗi extents mới sẽ tăng lên một cách tự động với đơn vị là 10 KB hoặc 10 MB.

- Các tệp dữ liệu thường được nhóm lại thành các nhóm tệp hoặc không gian bảng. Một không gian bảng (**table space**) hoặc nhóm tệp (**file group**) là một nhóm logic của nhiều tệp dữ liệu lưu trữ dữ liệu có các đặc điểm tương tự. Ví dụ, bạn có thể có một không gian bảng hệ thống nơi dữ liệu bảng từ điển được lưu trữ, một không gian bảng dành cho dữ liệu bảng do người dùng tạo ra, một không gian bảng dành cho lưu trữ tất cả các chỉ mục và một không gian bảng tạm thời để thực hiện các sắp xếp tạm thời, nhóm và các thao tác tương tự. Mỗi khi bạn tạo một cơ sở dữ liệu mới, hệ quản trị cơ sở dữ liệu tự động tạo ra một tập hợp tối thiểu các không gian bảng.



- Bộ đệm dữ liệu (**data cache**), hoặc bộ đệm đệm (**buffer cache**), là một khu vực bộ nhớ được dành riêng và được chia sẻ, lưu trữ các khối dữ liệu vừa được truy cập nhất trong RAM. Dữ liệu đọc từ các tệp dữ liệu được lưu trữ trong bộ đệm dữ liệu sau khi dữ liệu đã được đọc hoặc trước khi dữ liệu được ghi vào các tệp dữ liệu. Bộ đệm dữ liệu cũng lưu trữ dữ liệu trong bảng từ điển hệ thống và nội dung của các chỉ mục.



- Bộ đệm SQL (**SQL cache**), hoặc bộ đệm thủ tục (**procedure cache**), là một khu vực bộ nhớ được dành riêng và được chia sẻ, lưu trữ các câu lệnh SQL hoặc các thủ tục PL/SQL vừa được thực thi gần đây, bao gồm các triggers và hàm. Bộ đệm SQL không lưu trữ các câu lệnh SQL do người dùng cuối viết. Thay vào đó, bộ đệm SQL lưu trữ phiên bản "đã xử lý" của SQL sẵn sàng để thực thi bởi DBMS.
- Để làm việc với dữ liệu, DBMS phải truy xuất dữ liệu từ lưu trữ cố định và đặt nó trong RAM. Nói cách khác, dữ liệu được truy xuất từ các tệp dữ liệu và đặt trong bộ đệm dữ liệu.
- Để di chuyển dữ liệu từ lưu trữ cố định (tệp dữ liệu) vào RAM (bộ đệm dữ liệu), DBMS phát ra các yêu cầu I/O và đợi phản hồi. Một yêu cầu đọc I/O là một hoạt động truy xuất dữ liệu cấp thấp đọc hoặc ghi dữ liệu từ và đến các thiết bị máy tính, chẳng hạn như bộ nhớ, ổ đĩa cứng, hình ảnh và máy in. Lưu ý rằng hoạt động đọc ổ đĩa I/O trả về một khối đĩa vật lý toàn bộ, thường chứa nhiều hàng, từ lưu trữ cố định vào bộ đệm dữ liệu, ngay cả khi bạn chỉ sử dụng một thuộc tính từ chỉ một hàng. Kích thước khối đĩa vật lý phụ thuộc vào hệ điều hành và có thể là 4K, 8K, 16K, 32K, 64K hoặc thậm chí lớn hơn. Hơn nữa, tùy theo tình huống, một DBMS có thể phát ra yêu cầu đọc một khối đơn hoặc yêu cầu đọc nhiều khối.
- Làm việc với dữ liệu trong bộ đệm dữ liệu nhanh hơn nhiều lần so với làm việc với dữ liệu trong các tệp dữ liệu vì DBMS không cần phải chờ ổ đĩa cứng để truy xuất dữ liệu; không cần thực hiện các hoạt động I/O đĩa cứng để làm việc trong bộ đệm dữ liệu.

### **Extents**

Trong môi trường hệ quản trị cơ sở dữ liệu (DBMS), "mở rộng tự động" đề cập đến khả năng của các tệp dữ liệu tự động mở rộng kích thước một cách tự động bằng các đơn vị tăng trước được định sẵn.

### **Table space (file group)**

Trong một hệ quản trị cơ sở dữ liệu (DBMS), "không gian bảng" là một không gian lưu trữ logic được sử dụng để nhóm các dữ liệu có liên quan. Nó cũng được gọi là "nhóm tệp."

### **Data cache (buffer cache)**

Một khu vực bộ nhớ được chia sẻ và dành riêng, lưu trữ các khối dữ liệu vừa được truy cập gần đây trong RAM. Nó cũng được gọi là "bộ đệm dữ liệu" hoặc "bộ đệm đệm."

### **SQL cache (procedure cache)**

Một khu vực bộ nhớ được chia sẻ và dành riêng, lưu trữ các câu lệnh SQL hoặc các thủ tục PL/SQL vừa được thực thi gần đây, bao gồm các triggers và functions. Nó cũng được gọi là "bộ đệm thủ tục" hoặc "SQL cache."

### **Input/Output (I/O) request**

Một hoạt động truy xuất dữ liệu cấp thấp đọc hoặc ghi dữ liệu từ và đến các thiết bị máy tính.

- Hầu hết các hoạt động tinh chỉnh hiệu suất tập trung vào việc giảm thiểu số lần thực hiện các hoạt động I/O vì sử dụng các hoạt động I/O nhiều lần chậm hơn nhiều so với việc đọc dữ liệu từ bộ đệm dữ liệu. Ví dụ, vào thời điểm viết này, thời gian truy cập RAM dao động từ 5 đến 70 nanosecond, trong khi thời gian truy cập ổ đĩa cứng từ 5 đến 15 mili giây và thời gian truy cập SSD dao động từ 35 đến 100 micro giây. Điều này có nghĩa là ổ đĩa cứng chậm hơn RAM nhiều lần.

Hình 11.1 cũng minh họa một số quy trình DBMS điển hình. Mặc dù số lượng quy trình và tên của chúng có thể thay đổi từ nhà cung cấp sang nhà cung cấp, chức năng tương tự. Các quy trình sau được đại diện trong Hình 11.1:

- *Listener*: Quy trình nghe chờ yêu cầu từ các máy khách và xử lý việc xử lý các yêu cầu SQL cho các quy trình DBMS khác. Khi yêu cầu được nhận, quy trình nghe chuyển yêu cầu đó đến quy trình người dùng thích hợp.
- *User*: DBMS tạo một quy trình người dùng để quản lý từng phiên làm việc của mỗi máy khách. Do đó, khi bạn đăng nhập vào DBMS, bạn sẽ được gán một quy trình người dùng. Quy trình này xử lý tất cả các yêu cầu bạn gửi đến máy chủ. Có nhiều quy trình người dùng - ít nhất là một quy trình cho mỗi người dùng đăng nhập.
- *Scheduler*: Quy trình lập lịch tổ chức việc thực hiện đồng thời các yêu cầu SQL. (Xem Chương 10, Quản lý Giao dịch và Kiểm soát Đồng thời.)
- *Lock manager*: Quy trình quản lý khóa quản lý tất cả các khóa được đặt trên các đối tượng cơ sở dữ liệu, bao gồm các trang đĩa. (Xem Chương 10.)
- *Optimizer*: Quy trình tối ưu hóa phân tích các truy vấn SQL và tìm cách truy cập dữ liệu hiệu quả nhất. Bạn sẽ tìm hiểu thêm về quy trình này sau trong chương.

#### **Automatic query optimization**

Một phương pháp mà một Hệ quản trị cơ sở dữ liệu (DBMS) tìm đường truy cập hiệu quả nhất cho việc thực thi một truy vấn.

#### **Manual query optimization**

Chế độ hoạt động yêu cầu người dùng cuối hoặc người lập trình xác định đường truy cập cho việc thực thi của một truy vấn.

#### **Static query optimization**

Chế độ tối ưu hóa truy vấn trong đó đường truy cập đến cơ sở dữ liệu được xác định trước tại thời điểm biên dịch.

#### **Dynamic query optimization**

Một kỹ thuật tối ưu hóa truy vấn sử dụng thông tin thống kê về cơ sở dữ liệu. Sau đó, DBMS sử dụng thông tin thống kê này để xác định chiến lược truy cập tốt nhất.

## 11-1c Database Query Optimization Modes

Hầu hết các thuật toán được đề xuất cho việc tối ưu hóa truy vấn dựa trên hai nguyên tắc:

- Lựa chọn thứ tự thực hiện tối ưu để đạt được thời gian thực thi nhanh nhất.
- Lựa chọn các trang web hoặc vị trí cần truy cập để giảm thiểu chi phí giao tiếp.

Trong các nguyên tắc đó, một thuật toán tối ưu hóa truy vấn có thể được đánh giá dựa trên chế độ hoạt động hoặc thời gian tối ưu hóa của nó.

Các chế độ hoạt động có thể được phân loại thành thủ công hoặc tự động. Tối ưu hóa truy vấn tự động có nghĩa là Hệ quản trị cơ sở dữ liệu (DBMS) tìm đường truy cập hiệu quả về chi phí mà không cần sự can thiệp của người dùng. Tối ưu hóa truy vấn thủ công yêu cầu người dùng cuối hoặc người lập trình phải chọn và lên lịch tối ưu hóa. Tối ưu hóa

truy vấn tự động rõ ràng là lựa chọn hơn từ quan điểm của người dùng cuối, nhưng chi phí của sự tiện lợi đó là sự gia tăng của overhead nó tạo ra cho DBMS.

Các thuật toán tối ưu hóa truy vấn cũng có thể được phân loại dựa trên thời điểm tối ưu hóa. Trong phân loại thời gian này, các thuật toán tối ưu hóa truy vấn có thể là tĩnh hoặc động.

- Tối ưu hóa truy vấn (**static query optimization**) tĩnh diễn ra tại thời điểm biên dịch. Nói cách khác, chiến lược tối ưu tốt nhất được chọn khi truy vấn được biên dịch bởi DBMS. Điều này thường xảy ra khi các câu lệnh SQL được nhúng trong các ngôn ngữ lập trình thủ tục như C# hoặc Visual Basic .NET. Khi chương trình được gửi đến DBMS để biên dịch, nó tạo kế hoạch cần thiết để truy cập cơ sở dữ liệu. Khi chương trình được thực thi, DBMS sử dụng kế hoạch đó để truy cập cơ sở dữ liệu.
- Tối ưu hóa truy vấn động (**dynamic query optimization**) diễn ra tại thời điểm thực thi. Chiến lược truy cập cơ sở dữ liệu được xác định khi chương trình được thực thi. Do đó, chiến lược truy cập được xác định động bởi DBMS tại thời điểm chạy, sử dụng thông tin mới nhất về cơ sở dữ liệu. Mặc dù tối ưu hóa truy vấn động hiệu quả, chi phí của nó là sự gia tăng của hiệu suất xử lý thời gian chạy. Chiến lược tốt nhất được xác định mỗi khi truy vấn được thực hiện; điều này có thể xảy ra nhiều lần trong cùng một chương trình.

Cuối cùng, các kỹ thuật tối ưu hóa truy vấn có thể được phân loại dựa trên loại thông tin được sử dụng để tối ưu hóa truy vấn. Ví dụ, các truy vấn có thể dựa trên thuật toán dựa trên thống kê hoặc dựa trên quy tắc.

- Một thuật toán tối ưu hóa truy vấn dựa trên thống kê (**statistically based query optimization algorithm**) sử dụng thông tin thống kê về cơ sở dữ liệu. Thống kê cung cấp thông tin về các đặc điểm của cơ sở dữ liệu như kích thước, số bản ghi, thời gian truy cập trung bình, số yêu cầu được phục vụ và số người dùng có quyền truy cập. Thông tin thống kê này sau đó được sử dụng bởi Hệ quản trị cơ sở dữ liệu (DBMS) để xác định chiến lược truy cập tốt nhất.
- Trong tối ưu hóa dựa trên thống kê, một số DBMS cho phép đặt mục tiêu để chỉ định rằng tối ưu hóa cố gắng giảm thiểu thời gian để lấy hàng đầu hoặc hàng cuối. Giảm thiểu thời gian để lấy hàng đầu thường được sử dụng trong các hệ thống giao dịch và môi trường người dùng tương tác. Trong những trường hợp này, mục tiêu là hiển thị các hàng đầu tiên cho người dùng càng nhanh càng tốt. Sau đó, trong khi DBMS chờ người dùng cuộn qua dữ liệu, nó có thể truy xuất các hàng khác cho truy vấn. Đặt mục tiêu tối ưu hóa để giảm thiểu việc lấy hàng cuối thường được thực hiện trong SQL nhúng và bên trong các thủ tục lưu trữ. Trong những trường hợp này, quyền điều khiển sẽ không được trả lại cho ứng dụng gọi cho đến khi tất cả dữ liệu đã được truy xuất; do đó, quan trọng là lấy toàn bộ dữ liệu tới hàng cuối càng nhanh càng tốt để có thể trả lại quyền điều khiển.

#### **Statically based query optimization algorithm**

"Một kỹ thuật tối ưu hóa truy vấn sử dụng thông tin thống kê về cơ sở dữ liệu. Hệ quản trị cơ sở dữ liệu (DBMS) sau đó sử dụng thông tin này để xác định chiến lược truy cập tốt nhất."

#### **Dynamic statistical generation mode**

"Trong hệ quản trị cơ sở dữ liệu (DBMS), khả năng tự động đánh giá và cập nhật thống kê truy cập cơ sở dữ liệu sau mỗi hoạt động truy cập dữ liệu."

#### **Manual statistical generation mode**

Chế độ tạo thông tin truy cập dữ liệu thống kê cho việc tối ưu hóa truy vấn. Ở chế độ này, quản trị cơ sở dữ liệu (DBA) phải định kỳ chạy một quy trình để tạo thông tin truy cập dữ liệu - ví dụ, chạy lệnh RUNSTAT trong cơ sở dữ liệu IBM DB2.

#### **Rule-based query optimization algorithm**

Kỹ thuật tối ưu hóa truy vấn sử dụng các quy tắc và điểm đã được thiết lập để xác định cách tiếp cận tốt nhất để thực hiện một truy vấn.

#### **Database statistics**

Trong tối ưu hóa truy vấn, các đo lường về các đối tượng cơ sở dữ liệu, chẳng hạn như số hàng trong một bảng, số khối đĩa được sử dụng, độ dài hàng tối đa và trung bình, số cột trong mỗi hàng và số giá trị riêng biệt trong mỗi cột. Những thông tin thống kê như vậy cung cấp một bức tranh tổng quan về các đặc điểm của cơ sở dữ liệu.

- Thông tin thống kê được quản lý bởi DBMS và được tạo ra theo một trong hai chế độ khác nhau: động hoặc thủ công. Trong chế độ tạo thống kê động (**dynamic statical generation mode**), DBMS tự động đánh giá và cập nhật thông tin thống kê sau mỗi hoạt động truy cập dữ liệu. Trong chế độ tạo thống kê thủ công (**manual statical generation mode**), thông tin thống kê phải được cập nhật định kỳ thông qua tiện ích do người dùng chọn, chẳng hạn như lệnh RUNSTAT của IBM, được sử dụng bởi các DBMS DB2.
- Một thuật toán tối ưu hóa truy vấn dựa trên quy tắc (**rule-based query algorithm**) dựa trên một tập hợp các quy tắc do người dùng cuối hoặc quản trị cơ sở dữ liệu xác định để xác định chiến lược truy cập truy vấn tốt nhất. Các quy tắc này được nhập bởi người dùng cuối hoặc quản trị cơ sở dữ liệu và thường là tổng quan về bản chất.

Vì thông tin thống kê về cơ sở dữ liệu đóng một vai trò quan trọng trong tối ưu hóa truy vấn, chủ đề này sẽ được khám phá chi tiết hơn trong phần tiếp theo.

## 11-1d Database Statistics

Một quy trình DBMS khác có vai trò quan trọng trong tối ưu hóa truy vấn là việc thu thập thông tin thống kê cơ sở dữ liệu. Thuật ngữ "thống kê cơ sở dữ liệu ()" đề cập đến một số đo lường về các đối tượng cơ sở dữ liệu, chẳng hạn số lượng bộ xử lý sử dụng, tốc độ bộ xử lý và không gian tạm thời có sẵn. Những thông tin thống kê như vậy cung cấp một cái nhìn tổng quan về các đặc điểm của cơ sở dữ liệu.

Như bạn sẽ tìm hiểu sau trong chương này, DBMS sử dụng thông tin thống kê này để đưa ra quyết định quan trọng về việc cải thiện hiệu suất xử lý truy vấn. Thống kê cơ sở dữ liệu có thể được thu thập thủ công bởi DBA hoặc tự động bởi DBMS. Ví dụ, nhiều nhà cung cấp DBMS hỗ trợ lệnh ANALYZE trong SQL để thu thập thông tin thống kê. Ngoài ra, nhiều nhà cung cấp có các quy trình riêng để thu thập thông tin thống kê. Ví dụ, DB2 của IBM sử dụng thủ tục RUNSTATS, trong khi SQL Server của Microsoft sử dụng thủ tục UPDATE STATISTICS và cung cấp các tùy chọn Auto-Update và Auto-Create Statistics trong các tham số khởi tạo của nó. Một số đo lường mẫu mà DBMS có thể thu thập về các đối tượng cơ sở dữ liệu khác nhau được hiển thị trong Bảng 11.2.



TABLE 11.2

## SAMPLE DATABASE STATISTICS MEASUREMENTS

| DATABASE OBJECT       | SAMPLE MEASUREMENTS                                                                                                                                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tables                | Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes |
| Indexes               | Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index            |
| Environment Resources | Logical and physical disk block size, location and size of data files, and number of extents per data file                                                                                                                 |

Nếu thông tin thống kê về đối tượng tồn tại, hệ quản trị cơ sở dữ liệu (DBMS) sẽ sử dụng chúng trong quá trình xử lý truy vấn. Hầu hết các DBMS mới hơn (như Oracle, MySQL, SQL Server và DB2) tự động thu thập thông tin thống kê; những DBMS khác đòi hỏi quản trị cơ sở dữ liệu (DBA) phải thu thập thông tin thống kê bằng tay. Để tạo ra thông tin thống kê về đối tượng cơ sở dữ liệu bằng tay, mỗi DBMS đều có các lệnh riêng của nó.

- Trong Oracle, sử dụng: **ANALYZE <TABLE/INDEX> object\_name COMPUTE STATISTICS;**

- Trong MySQL, sử dụng: **ANALYZE TABLE <tên\_bảng>;**

- Trong SQL Server, sử dụng: **UPDATE STATISTICS <tên\_đối\_tượng>;** trong đó tên đối tượng có thể là tên bảng hoặc tên view.

Ví dụ, để tạo thông tin thống kê cho bảng **VENDOR**, bạn có thể sử dụng

- Trong Oracle: **ANALYZE TABLE VENDOR COMPUTE STATISTICS;**

- Trong MySQL: **ANALYZE TABLE VENDOR;**

- Trong SQL Server: **UPDATE STATISTICS VENDOR;**

Khi bạn tạo thông tin thống kê cho một bảng, tất cả các chỉ mục liên quan cũng được phân tích. Tuy nhiên, bạn có thể tạo thông tin thống kê cho một chỉ mục cụ thể bằng cách sử dụng các lệnh sau, trong đó **VEND\_NDX** là tên chỉ mục:

- Trong Oracle: **ANALYZE INDEX VEND\_NDX COMPUTE STATISTICS;**

- Trong SQL Server: **UPDATE STATISTICS** <tên\_bảng> <tên\_chỉ\_mục>. Ví dụ lệnh có thể là: **UPDATE STATISTICS VENDOR VEND\_NDX**;

Thông tin thống kê cơ sở dữ liệu được lưu trữ trong bảng hệ thống được chỉ định đặc biệt trong các DBMS. Thông thường, việc tạo lại thông tin thống kê cho các đối tượng cơ sở dữ liệu, đặc biệt là đối tượng cơ sở dữ liệu mà thường xuyên thay đổi, là phổ biến. Ví dụ, nếu bạn có hệ thống quản lý cho thuê video, hệ thống của bạn có thể sử dụng một bảng RENTAL để lưu trữ thông tin về việc thuê video hàng ngày. Bảng RENTAL và các chỉ mục liên quan của nó sẽ thường xuyên được thêm mới và cập nhật khi bạn ghi lại thông tin về việc thuê và trả video hàng ngày. Do đó, thông tin thống kê của bảng RENTAL bạn tạo ra tuần trước có thể không phản ánh đúng tình trạng hiện tại của bảng. Thông tin thống kê càng cập nhật, khả năng hệ thống DBMS chọn cách nhanh nhất để thực hiện một truy vấn cụ thể càng cao.

Bây giờ, khi bạn đã hiểu về kiến trúc cơ bản của quá trình và cấu trúc bộ nhớ của hệ thống quản trị cơ sở dữ liệu (DBMS), và về sự quan trọng và thời điểm thu thập thông tin thống kê cơ sở dữ liệu bởi DBMS, bạn đã sẵn sàng để tìm hiểu cách DBMS xử lý yêu cầu truy vấn SQL.

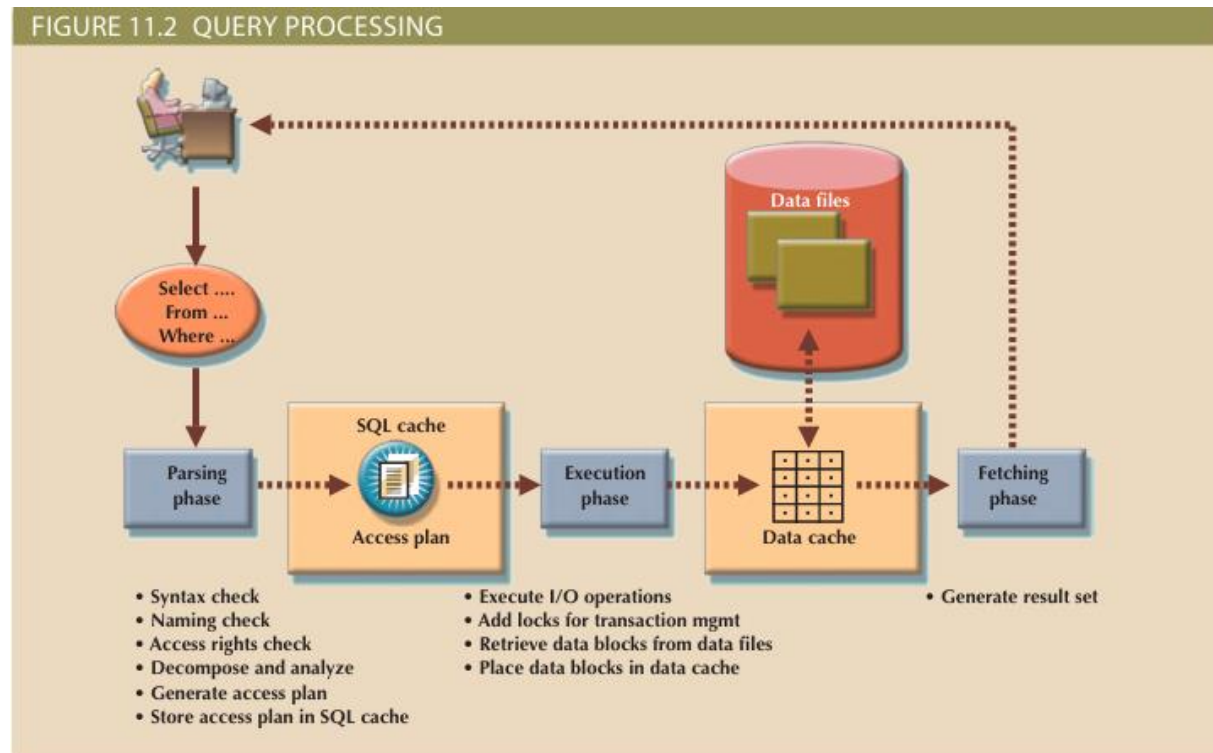
## 11-2 Query Processing

Khi máy chủ của hệ quản trị cơ sở dữ liệu (DBMS) nhận được câu lệnh SQL từ máy khách, quá trình xử lý câu truy vấn diễn ra ở ba giai đoạn cơ bản như sau:

1. **\*\*Phân tích (Parsing)\*\***: DBMS phân tích câu lệnh SQL và chọn kế hoạch truy cập/ thực hiện hiệu quả nhất.
2. **\*\*Thực thi (Execution)\*\***: DBMS thực thi câu lệnh SQL bằng cách sử dụng kế hoạch thực hiện đã được chọn.
3. **\*\*Truy xuất (Fetching)\*\***: DBMS truy xuất dữ liệu và gửi tập kết quả trở lại cho máy khách.

Quá trình xử lý các câu lệnh SQL DDL (như CREATE TABLE) khác biệt so với quá trình xử lý câu lệnh DML. Sự khác biệt nằm ở việc câu lệnh DDL thực sự cập nhật các bảng từ điển dữ liệu hoặc hệ thống, trong khi câu lệnh DML (SELECT, INSERT, UPDATE hoặc DELETE) chủ yếu thao tác trên dữ liệu của người dùng cuối. Hình 11.2

cho thấy các bước chung cần thiết cho việc xử lý truy vấn. Mỗi bước sẽ được thảo luận trong các phần tiếp theo.



## 11-2a SQL Parsing Phase

Quá trình tối ưu hóa (**optimization process**) bao gồm việc phân tách (parsing) truy vấn thành các đơn vị nhỏ hơn (**breaking down-parsing-the query into smaller units**) và biến đổi truy vấn SQL ban đầu thành một phiên bản SQL khác một chút so với mã SQL ban đầu, nhưng phiên bản này hoàn toàn tương đương và hiệu quả hơn. Tương đương hoàn toàn (*fully equivalent*) có nghĩa là kết quả truy vấn được tối ưu hóa luôn luôn giống với truy vấn ban đầu. Hiệu quả hơn có nghĩa là truy vấn được tối ưu hóa thường thực thi nhanh hơn so với truy vấn ban đầu (lưu ý rằng "gần như luôn luôn (*almost always*)" thực thi nhanh hơn vì nhiều yếu tố ảnh hưởng đến hiệu suất của cơ sở dữ liệu, như đã được giải thích trước đó. Những yếu tố này bao gồm mạng, tài nguyên của máy tính khách và các truy vấn khác đang chạy cùng lúc trong cùng một cơ sở dữ liệu). Để xác định cách thực hiện truy vấn hiệu quả nhất, hệ quản trị cơ sở dữ liệu (DBMS) có thể sử dụng thông tin thống kê về cơ sở dữ liệu mà bạn đã tìm hiểu trước đây.

Các hoạt động phân tích SQL được thực hiện bởi trình tối ưu hóa truy vấn (**query optimizer**), người phân tích truy vấn SQL và tìm cách hiệu quả nhất để truy cập dữ liệu.



Quá trình này là giai đoạn tốn thời gian nhất trong xử lý truy vấn. Phân tích một truy vấn SQL yêu cầu một số bước, trong đó truy vấn SQL được:

- Kiểm tra tính đúng cú pháp.
- Kiểm tra với từ điển dữ liệu để đảm bảo rằng tên bảng và tên cột là chính xác.
- Kiểm tra với từ điển dữ liệu để đảm bảo rằng người dùng có quyền truy cập hợp lệ.
- Phân tích và phân rã thành các thành phần nguyên tử hơn.
- Được tối ưu hóa thông qua biến đổi thành một phiên bản SQL hoàn toàn tương đương nhưng hiệu quả hơn.
- Được chuẩn bị cho việc thực hiện bằng cách xác định kế hoạch thực hiện hoặc kế hoạch truy cập hiệu quả nhất.

Sau khi câu lệnh SQL đã được biến đổi, hệ quản trị cơ sở dữ liệu (DBMS) tạo ra điều gọi thông thường là kế hoạch truy cập hoặc kế hoạch thực hiện. Kế hoạch truy cập (**access plan**) là kết quả của việc phân tách một câu lệnh SQL; nó chứa chuỗi các bước mà DBMS sẽ sử dụng để thực hiện truy vấn và trả kết quả một cách hiệu quả nhất. Đầu tiên, DBMS kiểm tra xem một kế hoạch truy cập đã tồn tại cho truy vấn trong bộ đệm SQL chưa. Nếu có, DBMS tái sử dụng kế hoạch truy cập để tiết kiệm thời gian. Nếu không có, trình tối ưu hóa đánh giá các kế hoạch khác nhau và sau đó quyết định sử dụng các chỉ mục nào và cách thực hiện phép nối tốt nhất. Kế hoạch truy cập được lựa chọn cho truy vấn sau đó được đặt trong bộ đệm SQL và s (future reuse)

Kế hoạch truy cập là đặc thù cho từng hệ quản trị cơ sở dữ liệu (DBMS) và dịch câu truy vấn

SQL của máy khách thành chuỗi các hoạt động I/O phức tạp cần thiết để đọc dữ liệu từ các tệp dữ liệu vật lý và tạo ra tập kết quả. Bảng 11.3 minh họa một số hoạt động I/O cho hệ quản trị cơ sở dữ liệu Oracle. Hầu hết các DBMS thực hiện các loại hoạt động tương tự khi truy cập và thao tác các tập dữ liệu.

#### **Access plan**

Một tập hướng dẫn được tạo ra tại thời điểm biên dịch ứng dụng, được tạo và quản lý bởi hệ quản trị cơ sở dữ liệu (DBMS). Kế hoạch truy cập xác định trước cách mà truy vấn của ứng dụng sẽ truy cập cơ sở dữ liệu khi chạy.

Trong Bảng 11.3, hãy lưu ý rằng việc truy cập bảng bằng cách sử dụng một số nhận dạng hàng (row ID) là phương pháp nhanh nhất. Một số nhận dạng hàng là một số nhận dạng duy nhất cho mỗi hàng được lưu trữ trong bộ nhớ lưu trữ cố định; nó có thể được sử dụng để truy cập trực tiếp hàng đó. Theo cách tư duy, một số nhận dạng hàng tương tự như một phiếu bạn nhận khi bạn đậu xe ở một bãi đỗ xe sân bay. Phiếu đậu xe chứa thông tin về số khu vực và số lô. Sử dụng thông tin đó, bạn có thể đi trực tiếp đến xe của mình mà không cần tìm kiếm từng khu vực và lô.

**TABLE 11.3**

**SAMPLE DBMS ACCESS PLAN I/O OPERATIONS**

| OPERATION             | DESCRIPTION                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------|
| Table scan (full)     | Reads the entire table sequentially, from the first row to the last, one row at a time (slowest)                      |
| Table access (row ID) | Reads a table row directly, using the row ID value (fastest)                                                          |
| Index scan (range)    | Reads the index first to obtain the row IDs and then accesses the table rows directly (faster than a full table scan) |
| Index access (unique) | Used when a table has a unique index in a column                                                                      |
| Nested loop           | Reads and compares a set of values to another set of values, using a nested loop style (slow)                         |
| Merge                 | Merges two data sets (slow)                                                                                           |
| Sort                  | Sorts a data set (slow)                                                                                               |

## 11-2b SQL Execution Phase

Trong giai đoạn này, tất cả các hoạt động I/O được chỉ ra trong kế hoạch thực hiện được thực thi. Khi kế hoạch thực hiện được chạy, các khóa phù hợp - nếu cần - được thu thập cho dữ liệu cần truy cập, và dữ liệu được lấy ra từ các tệp dữ liệu và đặt trong bộ nhớ dữ liệu của hệ quản trị cơ sở dữ liệu (DBMS). Tất cả các lệnh quản lý giao dịch được xử lý trong các giai đoạn phân tích và thực hiện của quá trình xử lý truy vấn.

## 11-2c SQL Fetching Phase

Sau khi các giai đoạn phân tích và thực hiện hoàn tất, tất cả các hàng phù hợp với các điều kiện đã được xác định sẽ được truy xuất, sắp xếp, nhóm và tổng hợp (nếu cần). Trong giai đoạn lấy dữ liệu, các hàng của tập kết quả của truy vấn được trả về cho máy khách. Hệ quản trị cơ sở dữ liệu (DBMS) có thể sử dụng không gian bảng tạm thời để lưu trữ dữ liệu tạm thời. Trong giai đoạn này, máy chủ cơ sở dữ liệu điều phối việc chuyển động các hàng kết quả từ bộ nhớ máy chủ đến bộ nhớ máy khách. Ví dụ, một

tập kết quả truy vấn cụ thể có thể chứa 9,000 hàng; máy chủ sẽ gửi 100 hàng đầu tiên đến máy khách và sau đó chờ máy khách yêu cầu tập hàng tiếp theo, cho đến khi toàn bộ tập kết quả được gửi đến máy khách.

## 11-2d Query Processing Bottlenecks

Mục tiêu chính của xử lý truy vấn là thực hiện một truy vấn cụ thể một cách nhanh nhất có thể với ít tài nguyên nhất. Như bạn đã thấy, việc thực hiện một truy vấn đòi hỏi hệ quản trị cơ sở dữ liệu (DBMS) phải phân rã truy vấn thành một chuỗi các hoạt động I/O phụ thuộc lẫn nhau để thực hiện một cách hợp tác. Truy vấn càng phức tạp, thì các hoạt động càng phức tạp, điều này đồng nghĩa với việc nút cổ càng có khả năng xảy ra. Một nút cổ trong xử lý truy vấn (**query processing bottleneck**) là sự trễ được giới thiệu trong việc xử lý một hoạt động I/O gây chậm hệ thống tổng thể. Tương tự, càng nhiều thành phần hệ thống, càng nhiều sự giao tiếp được yêu cầu giữa các thành phần, làm tăng khả năng nút cổ. Trong một hệ quản trị cơ sở dữ liệu (DBMS), năm thành phần thường gây ra nút cổ:

- **CPU.** Sức mạnh xử lý CPU của DBMS nên phù hợp với khối lượng công việc dự kiến của hệ thống. Sử dụng CPU cao có thể chỉ ra rằng tốc độ xử lý của bộ vi xử lý quá chậm so với khối lượng công việc được thực hiện. Tuy nhiên, việc sử dụng CPU nặng còn có thể do những yếu tố khác như một thành phần bị lỗi, không đủ RAM (CPU tiêu tốn quá nhiều thời gian để trao đổi các khối bộ nhớ), trình điều khiển thiết bị viết kém, hoặc quá trình không mong muốn. Nút cổ CPU sẽ ảnh hưởng không chỉ đến DBMS mà còn đến tất cả các quy trình đang chạy trong hệ thống.
- **RAM.** DBMS cấp phát bộ nhớ cho mục đích cụ thể, chẳng hạn như bộ nhớ đệm dữ liệu và bộ nhớ đệm SQL. RAM phải được chia sẻ giữa tất cả các quy trình đang chạy, bao gồm hệ điều hành và DBMS. Nếu không có đủ RAM, việc chuyển dữ liệu giữa các thành phần đang cạnh tranh để có RAM khan hiếm có thể tạo nút cổ.
- **Ổ cứng (hard disk).** Những nguyên nhân thông thường gây nút cổ khác là tốc độ ổ cứng và tỷ lệ truyền dữ liệu. Công nghệ lưu trữ ổ cứng hiện tại cho phép dung lượng lưu trữ lớn hơn so với quá khứ; tuy nhiên, không gian ổ cứng được sử dụng cho nhiều mục đích hơn chỉ để lưu trữ dữ liệu của người dùng cuối. Hệ điều hành hiện tại cũng sử dụng ổ cứng cho bộ nhớ ảo, có nghĩa là sao chép các phần của RAM lưu vào ổ cứng khi cần để giải phóng RAM cho các nhiệm vụ quan trọng hơn. Do đó, dung lượng lưu trữ ổ cứng lớn hơn và tốc độ truyền dữ liệu nhanh hơn sẽ làm giảm khả năng xảy ra nút cổ.

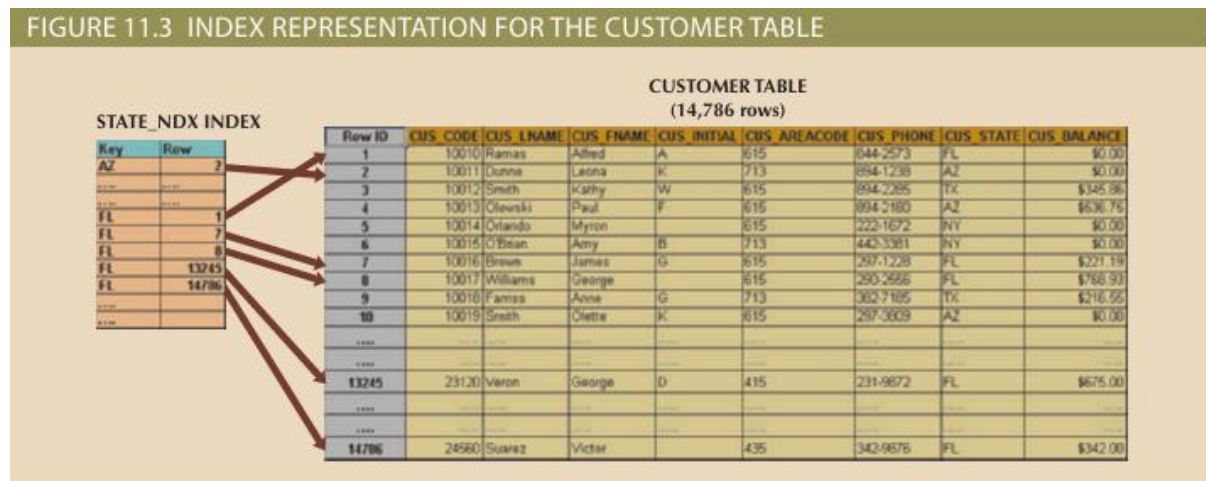
- *Mạng (network)*. Trong môi trường cơ sở dữ liệu, máy chủ cơ sở dữ liệu và các máy khách được kết nối thông qua mạng. Tất cả mạng đều có một lượng băng thông giới hạn được chia sẻ giữa tất cả các máy khách. Khi nhiều nút mạng truy cập mạng cùng một lúc, khả năng xảy ra nút cổ là rất cao.
- *Mã ứng dụng (application code)*. Không phải tất cả nút cổ được gây ra bởi tài nguyên phần cứng hạn chế. Hai nguồn gốc nút cổ phổ biến nhất là mã ứng dụng kém và cơ sở dữ liệu thiết kế kém. Mã ứng dụng kém có thể được cải thiện bằng các kỹ thuật tối ưu hóa mã, miễn là thiết kế cơ sở dữ liệu cơ bản là ổn định. Tuy nhiên, không có bất kỳ lượng mã lập trình nào có thể khiến cho cơ sở dữ liệu thiết kế kém hoạt động tốt hơn.

Những nút cổ là kết quả của nhiều giao dịch cơ sở dữ liệu cạnh tranh để sử dụng các tài nguyên cơ sở dữ liệu (CPU, RAM, ổ cứng, chỉ mục, khóa, bộ đệm, v.v.). Như bạn đã học trong chương này trước đó, hệ quản trị cơ sở dữ liệu (DBMS) sử dụng nhiều thành phần và cấu trúc để thực hiện các hoạt động của nó, chẳng hạn như các quy trình, bộ đệm, khóa, không gian bảng, chỉ mục và tệp nhật ký. Những tài nguyên này được sử dụng bởi tất cả các giao dịch thực thi trên cơ sở dữ liệu và do đó, các giao dịch thường cạnh tranh để sử dụng các tài nguyên này. Bởi vì hầu hết (nếu không phải tất cả) các giao dịch làm việc với các hàng dữ liệu trong các bảng, một trong những nút cổ phổ biến nhất là do các giao dịch cạnh tranh để truy cập cùng các hàng dữ liệu. Một nguồn cạnh tranh phổ biến khác là cho các tài nguyên bộ nhớ chia sẻ, đặc biệt là bộ đệm và khóa chia sẻ. Để tăng tốc các hoạt động cập nhật dữ liệu, DBMS sử dụng bộ đệm để lưu trữ tạm thời dữ liệu. Đồng thời, để quản lý việc truy cập dữ liệu, DBMS sử dụng các khóa. Học cách tránh những nút cổ này và tối ưu hóa hiệu suất cơ sở dữ liệu là trọng tâm chính của chương này.

## 11-3 Indexes and Query Optimization

Các chỉ mục (indexes) rất quan trọng trong việc tăng tốc truy cập dữ liệu vì chúng tạo điều kiện thuận lợi cho tìm kiếm, sắp xếp, sử dụng các chức năng tổng hợp và thậm chí các phép toán nối (join). Sự cải thiện về tốc độ truy cập dữ liệu xảy ra vì chỉ mục là một tập hợp các giá trị đã được sắp xếp chứa khóa chỉ mục và các con trỏ. Các con trỏ là các định danh hàng (row IDs) cho các hàng thực tế trong bảng. Khái niệm về chỉ mục dữ liệu tương tự với một chỉ mục sách. Khi bạn sử dụng một chỉ mục sách, bạn tra cứu một từ, tương tự với khóa chỉ mục. Từ đó đi kèm với một hoặc nhiều số trang nơi từ được sử dụng; những số này tương tự các con trỏ.

Việc quét (scan) chỉ mục hiệu quả hơn so với việc quét toàn bộ bảng vì dữ liệu chỉ mục đã được sắp xếp trước và thường lượng dữ liệu ít hơn. Do đó, khi thực hiện tìm kiếm, hầu hết khi DBMS nên sử dụng chỉ mục để truy cập một bảng thay vì quét tất cả các hàng trong bảng một cách tuần tự. Ví dụ, Hình 11.3 cho thấy biểu đồ chỉ mục của một bảng **KHÁCH HÀNG (CUSTOMER)** có 14,786 hàng và chỉ mục **STATE\_NDX** trên thuộc tính **CUS\_STATE**.



Giả sử bạn gửi truy vấn sau đây:

```
SELECT          CUS_NAME, CUS_STATE
FROM            CUSTOMER
WHERE           CUS_STATE = 'FL'
```

Nếu không có chỉ mục, Hệ quản trị cơ sở dữ liệu (DBMS) sẽ thực hiện quét toàn bộ bảng và đọc tất cả 14.786 hàng dữ liệu của khách hàng. Giả sử chỉ mục **STATE\_NDX** được tạo (và phân tích), DBMS sẽ tự động sử dụng chỉ mục để tìm đến khách hàng đầu tiên có trạng thái là 'FL' và sau đó tiếp tục đọc tất cả các hàng dữ liệu **CUSTOMER** tiếp theo, sử dụng các định danh hàng (row IDs) trong chỉ mục như một hướng dẫn. Giả sử chỉ có năm hàng thỏa mãn điều kiện **CUS\_STATE = 'FL'**, sẽ có năm truy cập vào chỉ mục và năm truy cập vào dữ liệu, tổng cộng là 10 truy cập I/O. DBMS sẽ không cần đọc khoảng 14.776 yêu cầu I/O cho các hàng dữ liệu của khách hàng không thỏa mãn điều kiện. Điều này tiết kiệm một lượng lớn chu kỳ CPU!

Nếu chỉ mục quan trọng đến vậy, tại sao không tạo chỉ mục cho mỗi cột trong mỗi bảng? Câu trả lời đơn giản là không thực tế để làm như vậy. Tạo chỉ mục cho mỗi

cột trong mỗi bảng sẽ gây quá tải cho DBMS trong việc xử lý việc duyệt chỉ mục, đặc biệt nếu bảng có nhiều thuộc tính và hàng, hoặc cần nhiều lệnh chèn, cập nhật và xóa.

Một thước đo quyết định nhu cầu về chỉ mục là độ dễ dàng (data sparsity) của cột bạn muốn chỉ mục. Độ dễ dàng dữ liệu (**data sparsity**) đề cập đến số lượng giá trị khác nhau mà một cột có thể có. Ví dụ, một cột STU\_SEX trong bảng SINH VIÊN có thể có chỉ hai giá trị có thể, M hoặc F; do đó, cột này được gọi là có độ dễ dàng thấp. Ngược lại, cột STU\_DOB lưu trữ ngày tháng sinh của học sinh có thể có nhiều giá trị ngày tháng khác nhau; do đó, cột này có độ dễ dàng cao. Biết độ dễ dàng giúp bạn quyết định việc sử dụng chỉ mục có phù hợp hay không. Ví dụ, khi bạn thực hiện tìm kiếm trong một cột có độ dễ dàng thấp, bạn có thể đọc một phần lớn các hàng trong bảng bất kể; do đó, việc xử lý chỉ mục có thể là công việc không cần thiết. Trong phần 11-5, bạn sẽ tìm hiểu cách xác định khi nào nên sử dụng chỉ mục.

Hầu hết các hệ quản trị cơ sở dữ liệu (DBMS) thực hiện chỉ mục bằng cách sử dụng một trong các cấu trúc dữ liệu sau:

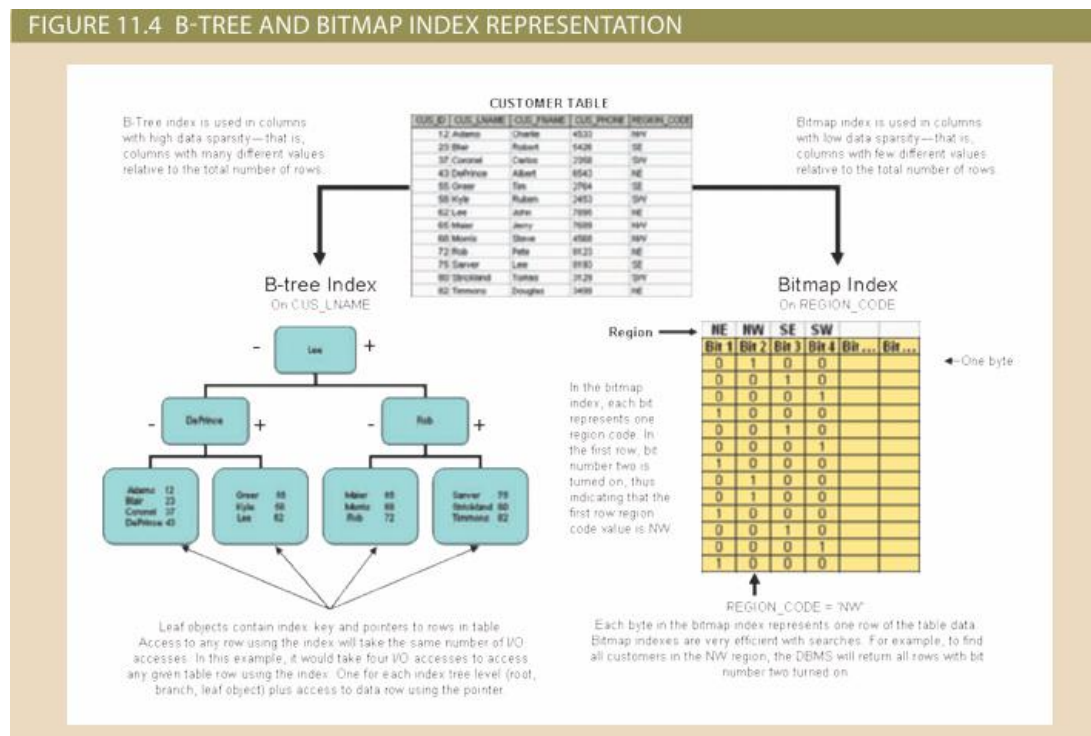
- Chỉ mục băm (**Hash index**): Chỉ mục băm dựa trên một danh sách đã được sắp xếp của các giá trị băm. Thuật toán băm được sử dụng để tạo ra một giá trị băm từ một cột khóa. Giá trị này trỏ đến một mục trong một bảng băm, từ đó trỏ đến vị trí thực tế của hàng dữ liệu. Loại chỉ mục này tốt cho các hoạt động tìm kiếm nhanh chóng dựa trên điều kiện bằng nhau, ví dụ: LNAME="Scott" và FNAME="Shannon".
- Chỉ mục B-tree (**B-tree index**): Chỉ mục B-tree là một cấu trúc dữ liệu đã được sắp xếp như một cây đảo ngược. (Xem Hình 11.4.) Cây chỉ mục B-tree được lưu trữ riêng biệt so với dữ liệu. Các lá ở cấp thấp nhất của chỉ mục B-tree chứa các con trỏ đến các hàng dữ liệu thực tế. Chỉ mục B-tree là "cân bằng tự động", có nghĩa rằng việc truy cập bất kỳ hàng nào trong chỉ mục mất khoảng thời gian tương tự. Đây là loại chỉ mục mặc định và phổ biến nhất được sử dụng trong cơ sở dữ liệu. Chỉ mục B-tree chủ yếu được sử dụng trong các bảng có giá trị cột lặp lại một số lần tương đối nhỏ.
- Chỉ mục Bitmap (**bitmap index**): Chỉ mục Bitmap sử dụng một mảng bit (0 và 1) để biểu thị sự tồn tại của một giá trị hoặc điều kiện. Chúng thường được sử dụng chủ yếu trong các ứng dụng lưu trữ dữ liệu có số lượng hàng lớn, trong đó một số lượng nhỏ giá trị cột lặp lại nhiều lần. (Xem Hình 11.4.) Chỉ mục Bitmap thường sử dụng ít không gian hơn so với chỉ mục B-tree vì chúng sử dụng các bit thay vì byte để lưu trữ dữ liệu của họ.



Dựa trên các đặc điểm chỉ mục trước đó, một người thiết kế cơ sở dữ liệu có thể xác định loại chỉ mục tốt nhất để sử dụng. Ví dụ, giả sử một bảng CUSTOMER có vài nghìn hàng. Bảng CUSTOMER có hai cột được sử dụng rộng rãi cho mục đích truy vấn: CUS\_LNAME, đại diện cho họ của khách hàng, và REGION\_CODE, có thể có một trong bốn giá trị (NE, NW, SW và SE). Dựa trên thông tin này, bạn có thể kết luận rằng:

- Bởi vì cột CUS\_LNAME chứa nhiều giá trị khác nhau và lặp lại một số lần tương đối ít so với tổng số hàng trong bảng, một chỉ mục B-tree sẽ được sử dụng.

FIGURE 11.4 B-TREE AND BITMAP INDEX REPRESENTATION



- Vì cột REGION\_CODE chỉ chứa một số giá trị khác nhau rất ít lặp lại nhiều lần so với tổng số hàng trong bảng, một chỉ mục bitmap sẽ được sử dụng. Hình 11.4 cho thấy biểu diễn chỉ mục B-tree và chỉ mục bitmap cho bảng CUSTOMER được sử dụng trong bài thảo luận trước đó.

Các DBMS thế hệ hiện tại đã đủ thông minh để xác định loại chỉ mục tốt nhất để sử dụng trong các trường hợp cụ thể, miễn là DBMS có cập nhật thống kê cơ sở dữ liệu. Bất kể chỉ mục nào được chọn, DBMS sẽ xác định kế hoạch tốt nhất để thực hiện một

**Data sparsity**  
Phân phối giá trị của cột hoặc số lượng giá trị khác nhau mà một cột có thể có.  
**Hard index**

truy vấn cụ thể. Phần tiếp theo sẽ hướng dẫn bạn qua một ví dụ đơn giản về loại lựa chọn mà trình tối ưu hóa truy vấn phải thực hiện

Một chỉ mục dựa trên một danh sách có thứ tự của các giá trị băm.

#### **B-tree index**

Một cấu trúc dữ liệu có thứ tự được tổ chức dưới dạng một cây đảo ngược.

#### **Bitmap index**

Một chỉ mục sử dụng một mảng bit (0 và 1) để biểu thị sự tồn tại của một giá trị hoặc điều kiện.

## 11-4 Optimizer Choices

Tối ưu hóa truy vấn là hoạt động trung tâm trong giai đoạn phân tích trong quá trình xử lý truy vấn. Trong giai đoạn này, hệ quản t

ri cơ sở dữ liệu (DBMS) phải chọn xem sẽ sử dụng những chỉ mục nào, cách thực hiện các phép nối (join), sử dụng bảng nào trước, và nhiều quyết định khác. Mỗi DBMS có thuật toán riêng để xác định cách hiệu quả nhất để truy cập dữ liệu. Trình tối ưu hóa truy vấn có thể hoạt động trong một trong hai chế độ sau:

- Trình tối ưu hóa dựa trên quy tắc (**rule-based optimizer**) sử dụng các quy tắc và điểm đã thiết lập trước để xác định cách tốt nhất để thực hiện một truy vấn. Các quy tắc gán một "chi phí cố định" cho mỗi phép thực hiện SQL; sau đó, các chi phí này được tổng hợp để tính toán chi phí của kế hoạch thực thi. Ví dụ, việc quét toàn bộ bảng có một chi phí cố định là 10, trong khi việc truy cập bảng bằng ID hàng có một chi phí cố định là 3.
- Trình tối ưu hóa dựa trên chi phí (**cost-based optimizer**) sử dụng các thuật toán tinh vi dựa trên thống kê về các đối tượng được truy cập để xác định cách tốt nhất để thực hiện một truy vấn. Trong trường hợp này, quá trình tối ưu hóa thêm các chi phí xử lý, chi phí I/O và chi phí tài nguyên (RAM và không gian tạm thời) để xác định tổng chi phí của kế hoạch thực hiện cụ thể.

Mục tiêu của trình tối ưu hóa là tìm cách thực hiện một truy vấn dưới dạng các cách thức thay thế, đánh giá "chi phí" của từng cách thức và sau đó chọn cách có chi phí thấp nhất. Để hiểu về chức năng của trình tối ưu hóa truy vấn, hãy xem xét một ví dụ đơn giản. Giả sử bạn muốn liệt kê tất cả các sản phẩm do một nhà cung cấp có trụ sở tại Florida cung cấp. Để có thông tin đó, bạn có thể viết truy vấn sau:

```

SELECT      P_CODE, P_DESCRIPT, P_PRICE, V_NAME, V_STATE
FROM        PRODUCT, VENDOR
WHERE       PRODUCT.V_CODE, = VENDOR.V_CODE

           AND VENDOR.V_STATE = 'FL'

```

Hơn nữa, giả sử thống kê cơ sở dữ liệu cho thấy những thông tin sau:

- Bảng PRODUCT có 7,000 hàng.
- Bảng VENDOR có 300 hàng.
- Mười nhà cung cấp đặt trụ sở tại Florida.
- Một nghìn sản phẩm đến từ các nhà cung cấp ở Florida.

Cần lưu ý rằng chỉ hai thông tin đầu tiên có sẵn cho trình tối ưu hóa. Hai thông tin thứ hai được giả định để minh họa những quyết định mà trình tối ưu hóa phải đưa ra. Dựa trên thông tin trong hai thông tin đầu tiên, trình tối ưu hóa sẽ cố gắng tìm cách hiệu quả nhất để truy cập dữ liệu. Yếu tố chính trong việc xác định kế hoạch truy cập hiệu quả nhất là chi phí I/O. (Hãy nhớ rằng DBMS luôn cố gắng giảm thiểu các thao tác I/O.) Bảng 11.4 cho thấy hai kế hoạch truy cập mẫu cho truy vấn trước đó và các chi phí I/O tương ứng của chúng.

Để làm ví dụ dễ hiểu hơn, cột I/O Operations và I/O Cost trong Bảng 11.4 chỉ ước tính số lần đọc đĩa I/O mà DBMS phải thực hiện.

**TABLE 11.4**

**COMPARING ACCESS PLANS AND I/O COSTS**

| PLAN     | STEP | OPERATION                                    | I/O OPERATIONS | I/O COST  | RESULTING SET ROWS | TOTAL I/O COST   |
|----------|------|----------------------------------------------|----------------|-----------|--------------------|------------------|
| <b>A</b> | A1   | Cartesian product (PRODUCT, VENDOR)          | 7,000 + 300    | 7,300     | 2,100,000          | 7,300            |
|          | A2   | Select rows in A1 with matching vendor codes | 2,100,000      | 2,100,000 | 7,000              | 2,107,300        |
|          | A3   | Select rows in A2 with V_STATE = 'FL'        | 7,000          | 7,000     | 1,000              | <b>2,114,300</b> |
| <b>B</b> | B1   | Select rows in VENDOR with V_STATE = 'FL'    | 300            | 300       | 10                 | 300              |
|          | B2   | Cartesian Product (PRODUCT, B1)              | 7,000 + 10     | 7,010     | 70,000             | 7,310            |
|          | B3   | Select rows in B2 with matching vendor codes | 70,000         | 70,000    | 1,000              | <b>77,310</b>    |

Vì tính đơn giản, giả sử không có bất kỳ chỉ mục nào và mỗi hàng đọc có chi phí I/O là 1. Ví dụ, ở Bước A1, DBMS phải tính tích hợp của PRODUCT và VENDOR. Để làm điều đó, DBMS phải đọc tất cả các hàng từ PRODUCT (7,000) và tất cả các hàng từ VENDOR (300), dẫn đến tổng cộng 7,300 thao tác I/O. Cùng một tính toán được thực hiện trong tất cả các bước. Trong Bảng 11.4, bạn có thể thấy cách Kế hoạch A có tổng chi phí I/O gần 30 lần cao hơn Kế hoạch B. Trong trường hợp này, trình tối ưu hóa sẽ chọn Kế hoạch B để thực hiện câu lệnh SQL.

Với điều kiện thích hợp, một số truy vấn có thể được giải quyết hoàn toàn chỉ bằng cách sử dụng một chỉ mục. Ví dụ, giả sử bạn đang sử dụng bảng SẢN PHẨM và chỉ mục P\_QOH\_NDX trong thuộc tính P\_QOH. Sau đó, một truy vấn như `SELECT MIN(P_QOH) FROM PRODUCT` có thể được giải quyết bằng cách chỉ đọc mục nhập đầu tiên trong chỉ mục P\_QOH\_NDX, mà không cần truy cập bất kỳ khối dữ liệu nào của bảng PRODUCT. (Hãy nhớ rằng chỉ mục mặc định theo thứ tự tăng dần.)

Bạn đã học trong Phần 11-3 rằng các cột có độ thưa thớt thấp không phải lựa chọn tốt cho việc tạo chỉ mục. Tuy nhiên, trong một số trường hợp, một chỉ mục trong một cột có độ thưa thớt thấp sẽ hữu ích. Ví dụ, giả sử bảng EMPLOYEE có 122,483 dòng. Nếu bạn muốn biết có bao nhiêu nhân viên nữ làm việc tại công ty, bạn sẽ viết một truy vấn như sau:

```
SELECT COUNT(EMP_SEX) FROM NHÂN VIÊN WHERE EMP_SEX = 'F';
```

## NOTE

Không phải tất cả các hệ quản trị cơ sở dữ liệu (DBMS) tối ưu hóa các truy vấn SQL cùng cách. Trong thực tế, Oracle tạo cú pháp truy vấn khác biệt so với các phương pháp được mô tả trong nhiều phần trong chương này. Hãy luôn đọc tài liệu để xem xét yêu cầu tối ưu hóa cho triển khai DBMS của bạn.

Nếu bạn không có chỉ mục cho cột EMP\_SEX, truy vấn sẽ phải thực hiện quét toàn bộ bảng để đọc tất cả các dòng EMPLOYEE và mỗi dòng đầy đủ bao gồm các thuộc tính bạn không cần. Tuy nhiên, nếu bạn có một chỉ mục trên EMP\_SEX, truy vấn có thể được trả lời chỉ bằng cách đọc dữ liệu chỉ mục, mà không cần truy cập dữ liệu nhân viên.

## 11-4a Using Hints to Affect Optimizer Choices

Mặc dù trình tối ưu hóa thường hoạt động rất tốt trong hầu hết các tình huống, tuy nhiên đôi khi trình tối ưu hóa có thể không chọn lựa kế hoạch thực thi tốt nhất. Hãy nhớ rằng trình tối ưu hóa đưa ra quyết định dựa trên thông kê hiện có. Nếu thông kê quá cũ, trình tối ưu hóa có thể không thực hiện công việc tốt trong việc chọn lựa kế hoạch thực thi tốt nhất. Ngay cả khi có thông kê hiện tại, sự lựa chọn của trình tối ưu hóa có thể không phải lúc nào cũng là lựa chọn hiệu quả nhất. Đôi khi người dùng cuối muốn thay đổi chế độ tối ưu hóa cho câu lệnh SQL hiện tại. Để làm điều đó, bạn cần sử dụng gợi ý trình tối ưu hóa. Gợi ý trình tối ưu (**optimizer hints**) hóa là các hướng dẫn đặc biệt cho trình tối ưu hóa được nhúng bên trong văn bản lệnh SQL. Bảng 11.5 tóm tắt một số gợi ý trình tối ưu hóa phổ biến nhất được sử dụng trong SQL tiêu chuẩn.

Bây giờ khi bạn đã quen với cách DBMS xử lý các truy vấn SQL, bạn có thể chuyển sự chú ý của mình đến một số lời khuyên lập trình SQL chung để tạo điều kiện thuận lợi cho công việc của trình tối ưu hóa truy vấn.

TABLE 11.5

| OPTIMIZER HINTS |                                                                                                                                                                                                                                                                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HINT            | USAGE                                                                                                                                                                                                                                                                                                                                                  |
| ALL_ROWS        | Instructs the optimizer to minimize the overall execution time—that is, to minimize the time needed to return all rows in the query result set. This hint is generally used for batch mode processes. For example:<br><pre>SELECT /*+ ALL_ROWS */ *<br/>FROM   PRODUCT<br/>WHERE  P_QOH &lt; 10;</pre>                                                 |
| FIRST_ROWS      | Instructs the optimizer to minimize the time needed to process the first set of rows—that is, to minimize the time needed to return only the first set of rows in the query result set. This hint is generally used for interactive mode processes. For example:<br><pre>SELECT /*+ FIRST_ROWS */ *<br/>FROM   PRODUCT<br/>WHERE  P_QOH &lt; 10;</pre> |
| INDEX(name)     | Forces the optimizer to use the P_QOH_NDX index to process this query. For example:<br><pre>SELECT /*+ INDEX(P_QOH_NDX) */ *<br/>FROM   PRODUCT<br/>WHERE  P_QOH &lt; 10</pre>                                                                                                                                                                         |

### 11-5 SQL Performance Tuning

Tối ưu hóa hiệu suất SQL được đánh giá từ quan điểm của khách hàng. Vì vậy, mục tiêu là để minh họa một số thực hành thông thường được sử dụng để viết mã SQL hiệu quả. Một vài lời cảnh báo là thích hợp:

- Hầu hết các hệ quản trị cơ sở dữ liệu quan hệ thế hệ mới hiện nay thực hiện tối ưu hóa truy vấn tự động tại đầu máy chủ.
- Hầu hết các kỹ thuật tối ưu hiệu suất SQL đều đặc thù cho hệ quản trị cơ sở dữ liệu và do đó, hiếm khi có thể dùng chung, ngay cả trên các phiên bản khác nhau của cùng một hệ quản trị cơ sở dữ liệu. Một phần của lý do cho hành vi này là sự tiến bộ liên tục trong các công nghệ cơ sở dữ liệu.

Điều này có nghĩa là bạn không cần phải lo lắng về cách một truy vấn SQL được viết vì DBMS sẽ luôn tối ưu nó? Không, bởi vì có rất nhiều sự cải thiện có thể được đạt được. (DBMS sử dụng các kỹ thuật tối ưu chung thay vì tập trung vào các kỹ thuật cụ thể được quy định bởi các hoàn cảnh đặc biệt của thực hiện truy vấn.) Một truy vấn SQL viết chưa tốt có thể gây đau đớn cho hệ thống cơ sở dữ liệu từ quan điểm hiệu suất. Phần lớn các vấn đề hiệu suất cơ sở dữ liệu hiện nay liên quan đến mã SQL viết chưa tốt. Do đó, mặc dù DBMS cung cấp các dịch vụ tối ưu hoá chung, một truy vấn viết kỹ lưỡng hầu như luôn có hiệu suất tốt hơn một truy vấn viết chưa tốt.

Mặc dù các lệnh điều chỉnh dữ liệu SQL bao gồm rất nhiều lệnh khác nhau như INSERT, UPDATE, DELETE và SELECT, hầu hết các khuyến nghị trong phần này liên quan đến việc sử dụng lệnh SELECT và đặc biệt là việc sử dụng chỉ mục và cách viết biểu thức điều kiện.

### 11-5a Index Selectivity

Chỉ mục là kỹ thuật quan trọng nhất được sử dụng trong tối ưu hiệu suất SQL. Chìa khóa là biết khi nào sử dụng index. Theo nguyên tắc chung, chỉ mục có khả năng được sử dụng trong các trường hợp sau:

- Khi một cột có chỉ mục xuất hiện độc lập trong các tiêu chí tìm kiếm của mệnh đề WHERE hoặc HAVING.
- Khi một cột có chỉ mục xuất hiện độc lập trong mệnh đề GROUP BY hoặc ORDER BY
- Khi một hàm MAX hoặc MIN được áp dụng cho một cột có chỉ mục
- Khi độ dày dữ liệu trên cột có chỉ mục là cao.

Chỉ mục là rất hữu ích khi bạn muốn chọn một tập con nhỏ của các hàng từ một bảng lớn dựa trên điều kiện cụ thể. Nếu một chỉ mục tồn tại cho cột được sử dụng trong lựa



chọn, hệ quản trị cơ sở dữ liệu có thể chọn sử dụng nó. Mục tiêu là tạo ra các chỉ mục với độ lựa chọn cao. [Index selectivity](#) là một thước đo cho khả năng sử dụng chỉ mục trong xử lý truy vấn. Dưới đây là một số hướng dẫn chung để tạo và sử dụng chỉ mục:

- Tạo chỉ mục cho mỗi thuộc tính đơn được sử dụng trong các mệnh đề WHERE, HAVING, ORDER BY hoặc GROUP BY. Nếu bạn tạo chỉ mục cho tất cả các thuộc tính đơn được sử dụng trong các điều kiện tìm kiếm, hệ quản trị cơ sở dữ liệu sẽ truy cập vào bảng bằng cách sử dụng chỉ mục quét thay vì quét toàn bộ bảng. Ví dụ, nếu bạn có một chỉ mục cho P\_PRICE, điều kiện  $P\_PRICE > 10,00$  có thể được giải quyết bằng cách truy cập vào chỉ mục thay vì quét tuần tự tất cả các hàng trong bảng và đánh giá P\_PRICE cho mỗi hàng. Chỉ mục cũng được sử dụng trong các biểu thức nối, chẳng hạn như trong  $CUSTOMER.CUS\_CODE = INVOICE.CUS\_CODE$ .
- Không sử dụng chỉ mục trong các bảng nhỏ hoặc các bảng có độ dày thấp. Hãy nhớ, các bảng nhỏ và các bảng có độ dày thấp không phải là những thứ giống nhau. Một điều kiện tìm kiếm trong bảng có độ dày thấp có thể trả về một số lượng lớn các hàng trong bảng bất kể, làm cho hoạt động chỉ mục quá tốn kém và làm cho quét toàn bộ bảng là một lựa chọn khả thi. Theo cùng một logic, không tạo chỉ mục cho các bảng có một số hàng và một số thuộc tính - trừ khi bạn phải đảm bảo sự tồn tại của các giá trị duy nhất trong một cột.
- Khai báo các khóa chính và khóa ngoại để trình tối ưu hóa có thể sử dụng các chỉ mục trong các hoạt động nối. Tất cả các nối tự nhiên và nối phong cách cũ sẽ được hưởng lợi nếu bạn khai báo các khóa chính và khóa ngoại, vì trình tối ưu hóa sẽ sử dụng các chỉ mục có sẵn trong lúc nối. (Việc khai báo một PK hoặc FK, chính là khóa chính hoặc khóa ngoại, sẽ tự động tạo ra một chỉ mục cho cột được khai báo.) Ngoài ra, vì cùng lý do đó, viết nối bằng cú pháp SQL JOIN là tốt hơn. (Xem chương 7, Giới thiệu ngôn ngữ truy vấn cấu trúc (SQL).)
- Khai báo chỉ mục trong các cột nối khác với PK hoặc FK. Nếu bạn thực hiện các hoạt động nối trên các cột khác ngoài khóa chính và khóa ngoại, bạn có thể tốt hơn khi khai báo chỉ mục trong những cột đó.

Bạn không phải lúc nào cũng có thể sử dụng chỉ mục để cải thiện hiệu suất. Ví dụ, khi sử dụng dữ liệu được hiển thị trong Bảng 11.6 trong phần tiếp theo, việc tạo một chỉ mục cho P\_MIN sẽ không giúp gì cho điều kiện tìm kiếm  $P\_QOH > P\_MIN * 1.10$ . Lý do là trong một số DBMS, chỉ mục bị bỏ qua khi bạn sử dụng các hàm trong các thuộc tính bảng. Tuy nhiên, các cơ sở dữ liệu lớn như Oracle, SQL Server và DB2 hiện đã

hỗ trợ chỉ mục dựa trên chức năng. Function-based index là một chỉ mục dựa trên một hàm hoặc biểu thức SQL cụ thể. Ví dụ, bạn có thể tạo một chỉ mục trên YEAR (INV\_DATE). Chỉ mục dựa trên chức năng đặc biệt hữu ích khi xử lý các thuộc tính có nguồn gốc. Ví dụ, bạn có thể tạo một chỉ mục trên EMP\_SALARY + EMP\_COMMISSION.

Bạn nên tạo bao nhiêu chỉ mục? Lưu ý rằng bạn không nên tạo chỉ mục cho mỗi cột trong bảng. Quá nhiều chỉ mục sẽ làm chậm các hoạt động INSERT, UPDATE và DELETE, đặc biệt là nếu bảng chứa hàng ngàn hàng. Hơn nữa, một số trình tối ưu truy vấn sẽ chỉ chọn một chỉ mục làm chỉ mục chính để thực hiện truy vấn, ngay cả khi truy vấn của bạn sử dụng các điều kiện trong nhiều cột có chỉ mục khác nhau. Chỉ mục nào được trình tối ưu hóa sử dụng? Nếu bạn sử dụng trình tối ưu hóa dựa trên chi phí, câu trả lời sẽ thay đổi theo thời gian khi các hàng mới được thêm vào hoặc xóa khỏi các bảng. Trong mọi trường hợp, bạn nên tạo chỉ mục trong tất cả các cột tìm kiếm và sau đó cho phép trình tối ưu hóa chọn. Quan trọng là liên tục đánh giá việc sử dụng chỉ mục - giám sát, kiểm tra, đánh giá và cải tiến nếu hiệu suất không đủ.

#### 11-5b Conditional Expressions

Một biểu thức có điều kiện thường được đặt trong các mệnh đề WHERE hoặc HAVING của một câu lệnh SQL. Còn được gọi là tiêu chí có điều kiện, một biểu thức có điều kiện giới hạn đầu ra của một truy vấn chỉ ở các hàng phù hợp với tiêu chí có điều kiện. Thông thường, các tiêu chí có điều kiện có dạng được hiển thị trong Bảng 11.6.

**TABLE 11.6**

#### **CONDITIONAL CRITERIA**

| OPERAND1  | CONDITIONAL OPERATOR | OPERAND2     |
|-----------|----------------------|--------------|
| P_PRICE   | >                    | 10.00        |
| V_STATE   | =                    | FL           |
| V_CONTACT | LIKE                 | Smith%       |
| P_QOH     | >                    | P_MIN * 1.10 |

Trong Bảng 11.6, lưu ý rằng một toán hạng có thể là:

- Tên cột đơn giản như P\_PRICE hoặc V\_STATE
- Định dạng hay hằng số như giá trị 10.00 hoặc văn bản 'FL'
- Biểu thức như P\_MIN \* 1.10

Hầu hết các kỹ thuật tối ưu truy vấn được đề cập dưới đây được thiết kế để giúp cho bộ tối ưu hóa có thể làm việc dễ dàng hơn. Các thực hành phổ biến sau được sử dụng để viết biểu thức có điều kiện hiệu quả trong mã SQL.

- Sử dụng cột đơn giản hoặc các giá trị hằng số như các toán hạng trong biểu thức có điều kiện – tránh sử dụng biểu thức có điều kiện với hàm khi có thể. So sánh nội dung của một cột đơn với một giá trị định dạng nhanh hơn so với so sánh với các biểu thức. Ví dụ,  $P\_PRICE > 10,00$  nhanh hơn  $P\_QOH > P\_MIN * 1.10$  vì hệ quản trị cơ sở dữ liệu phải đánh giá biểu thức  $P\_MIN * 1.10$  trước. Việc sử dụng các hàm trong biểu thức cũng làm tăng tổng thời gian thực thi truy vấn. Ví dụ, nếu điều kiện của bạn là  $UPPER(V\_NAME) = 'JIM'$ , hãy thử sử dụng  $V\_NAME = 'Jim'$  nếu tất cả các tên trong cột  $V\_NAME$  được lưu trữ với chữ in đúng cú pháp.

- So sánh trường số có tốc độ nhanh hơn so với so sánh trường ký tự, ngày và NULL. Trong điều kiện tìm kiếm, so sánh một thuộc tính số với một hằng số số là nhanh hơn so sánh một thuộc tính ký tự với hằng số ký tự. Nói chung, CPU xử lý so sánh số (nguyên và thập phân) nhanh hơn so sánh ký tự và so sánh ngày. Bởi vì chỉ mục không lưu trữ tham chiếu đến các giá trị null, các điều kiện NULL liên quan đến xử lý bổ sung và do đó có xu hướng là toán tử điều kiện chậm nhất trong tất cả các toán hạng điều kiện.

- So sánh bằng thường nhanh hơn so sánh không bằng. Ví dụ,  $P\_PRICE = 10.00$  được xử lý nhanh hơn vì hệ quản trị cơ sở dữ liệu có thể tìm kiếm trực tiếp bằng cách sử dụng chỉ mục trong cột. Nếu không có sự khớp chính xác, điều kiện được đánh giá là sai. Tuy nhiên, nếu bạn sử dụng một ký hiệu không bằng ( $>$ ,  $>=$ ,  $<$ ,  $<=$ ), hệ quản trị cơ sở dữ liệu phải thực hiện xử lý bổ sung để hoàn tất yêu cầu, vì hầu hết luôn luôn sẽ có nhiều giá trị "lớn hơn" hoặc "nhỏ hơn" trong chỉ mục hơn là giá trị "bằng". Ngoại trừ NULL, toán tử so sánh chậm nhất trong tất cả các toán hạng là LIKE với các ký tự đại diện, như trong  $V\_CONTACT LIKE "\%glo\%"$ . Hơn nữa, sử dụng ký hiệu "không bằng" ( $<>$ ) cho kết quả tìm kiếm chậm hơn, đặc biệt là khi sự thừa thớt của dữ liệu là cao – đó là khi có nhiều giá trị khác nhau hơn là những giá trị bằng nhau.

Một biểu thức có điều kiện thường được đặt trong các mệnh đề WHERE hoặc HAVING của một câu lệnh SQL. Còn được gọi là tiêu chí có điều kiện, một biểu thức có điều kiện giới hạn đầu ra của một truy vấn chỉ ở các hàng phù hợp với tiêu chí có điều kiện. Thông thường, các tiêu chí có điều kiện có dạng được hiển thị trong Bảng 11.6.

Khi viết truy vấn, thông thường sẽ hướng đến việc trả lời câu hỏi. Ví dụ, nếu người dùng cuối cùng cung cấp cho bạn đầu ra mẫu và yêu cầu bạn phải phù hợp với định dạng đầu ra đó, bạn phải viết SQL tương ứng. Để thực hiện công việc, bạn phải đánh giá cẩn thận các cột, bảng và

- Khi nào có thể, chuyển đổi các biểu thức có điều kiện để sử dụng các giá trị định dạng. Ví dụ, nếu điều kiện của bạn là  $P\_PRICE - 10 = 7$ , hãy chuyển nó thành  $P\_PRICE = 17$ . Ngoài ra, nếu bạn có một điều kiện tổng hợp như sau:

$P\_QOH < P\_MIN$  AND  $P\_MIN = P\_REORDER$  AND  $P\_QOH = 10$

hãy chuyển đổi nó để được đọc như sau:

$P\_QOH = 10$  AND  $P\_MIN = P\_REORDER$  AND  $P\_MIN > 10$

- Khi sử dụng nhiều biểu thức có điều kiện, viết điều kiện bằng chữ đầu tiên. Lưu ý rằng điều này được thực hiện trong ví dụ trước đó. Hãy nhớ, các điều kiện bằng toán tử bằng được xử lý nhanh hơn so với các điều kiện không bằng. Mặc dù hầu hết các RDBMS sẽ tự động làm điều này cho bạn, tập trung vào chi tiết này sẽ giảm bớt tác động lên trình tối ưu hóa truy vấn. Trình tối ưu hóa sẽ không phải làm điều mà bạn đã làm trước đó.

- Nếu bạn sử dụng nhiều điều kiện AND, hãy viết điều kiện có khả năng là sai nhất đầu tiên. Nếu sử dụng kỹ thuật này, hệ thống DBMS sẽ dừng việc đánh giá các điều kiện còn lại ngay khi nó tìm thấy biểu thức có điều kiện được đánh giá là sai. Hãy nhớ, để nhiều điều kiện AND được đánh giá là đúng, tất cả điều kiện phải được đánh giá là đúng. Nếu một trong các điều kiện được đánh giá là sai, toàn bộ các điều kiện sẽ được đánh giá là sai. Nếu sử dụng kỹ thuật này, hệ thống DBMS sẽ không phí thời gian đánh giá các điều kiện bổ sung không cần thiết. Tự nhiên, việc sử dụng kỹ thuật này gợi ý rằng bạn hiểu về sự thừa thớt của tập dữ liệu. Ví dụ, nhìn vào danh sách điều kiện sau:

$P\_PRICE > 10$  AND  $V\_STATE = 'FL'$

Nếu bạn biết rằng chỉ một số nhà cung cấp được đặt tại Florida, bạn có thể viết lại điều kiện này như sau:

$V\_STATE = 'FL'$  AND  $P\_PRICE > 10$

- Khi sử dụng nhiều điều kiện OR, hãy để điều kiện có khả năng đúng nhất đứng đầu. Bằng cách làm này, hệ thống DBMS sẽ dừng việc đánh giá các điều kiện còn lại ngay khi nó tìm thấy một biểu thức có điều kiện được đánh giá là đúng. Hãy nhớ rằng, để nhiều điều kiện OR được đánh giá là đúng, chỉ cần một trong các điều kiện được đánh giá là đúng.

- Khi nào có thể, hãy tránh sử dụng toán tử logic NOT. Tốt nhất là chuyển đổi một biểu thức SQL chứa một toán tử logic NOT thành một biểu thức tương đương. Ví dụ:

NOT (P\_PRICE > 10,00) có thể được viết thành P\_PRICE <= 10,00.

Ngoài ra, NOT (EMP\_SEX = 'M') có thể được viết thành EMP\_SEX = 'F'.

**Note:** Oracle không đánh giá các truy vấn theo cách được mô tả ở đây. Thay vào đó, Oracle đánh giá các điều kiện từ cuối cùng trở về đầu tiên.

## 11-6 Query Formulation

Truy vấn thường được viết để trả lời các câu hỏi. Ví dụ, nếu người dùng cuối yêu cầu bạn phải khớp định dạng đầu ra của một mẫu đầu ra, bạn phải viết SQL tương ứng. Để thực hiện công việc đó, bạn phải đánh giá cẩn thận các cột, bảng và các tính toán cần thiết để tạo ra đầu ra mong muốn. Để làm điều đó, bạn phải có một hiểu biết tốt về môi trường cơ sở dữ liệu và cơ sở dữ liệu sẽ là trung tâm của mã SQL của bạn.

Phần này tập trung vào các truy vấn SELECT vì chúng là các truy vấn mà bạn sẽ tìm thấy trong hầu hết các ứng dụng. Để đưa ra một truy vấn, bạn thường sẽ làm theo các bước sau:

1. Xác định cột và tính toán nào cần thiết. Bước đầu tiên cần được thực hiện để xác định rõ những giá trị dữ liệu mà bạn muốn trả lại. Bạn có muốn trả lại chỉ tên và địa chỉ, hay bạn cũng muốn bao gồm một vài tính toán? Hãy nhớ rằng tất cả các cột trong câu lệnh SELECT nên trả lại một giá trị duy nhất.

- a. Bạn có cần các biểu thức đơn giản? Ví dụ, bạn có cần nhân giá của sản phẩm với số lượng có sẵn để tạo ra tổng chi phí hàng tồn kho? Bạn có thể cần sử dụng một số hàm thuộc tính đơn – ví dụ, DATE(), SYSDATE(), hoặc ROUND().

- b. Bạn có cần các hàm tổng hợp? Nếu bạn cần tính toán tổng doanh số theo sản phẩm, bạn nên sử dụng một mệnh đề GROUP BY. Trong một số trường hợp, bạn có thể cần sử dụng một truy vấn phụ.

c. Xác định độ chi tiết của dữ liệu đầu vào được yêu cầu cho đầu ra của bạn. Đôi khi, bạn có thể cần tóm tắt dữ liệu không sẵn có trong bất kỳ bảng nào. Trong những trường hợp như vậy, bạn có thể xem xét phân tách truy vấn thành nhiều truy vấn phụ và lưu trữ các truy vấn phụ đó như các chế độ xem. Sau đó, bạn có thể tạo một truy vấn cấp cao hơn kết nối những chế độ xem đó và tạo ra đầu ra cuối cùng.

2. Xác định các bảng nguồn. Sau khi bạn biết cột gì cần, bạn có thể xác định các bảng nguồn được sử dụng trong truy vấn. Một số thuộc tính xuất hiện trong nhiều bảng. Trong những trường hợp đó, hãy cố gắng sử dụng số lượng bảng ít nhất trong truy vấn của bạn để giảm thiểu số lượng thao tác nối.

3. Xác định cách kết nối các bảng. Sau khi bạn biết những bảng bạn cần trong câu lệnh truy vấn của mình, bạn phải định danh đúng cách cách kết nối các bảng. Trong hầu hết các trường hợp, bạn sẽ sử dụng một loại kết nối inner join, nhưng trong một số trường hợp, bạn có thể cần sử dụng một kết nối outer join.

4. Xác định tiêu chí lựa chọn cần thiết. Hầu hết các truy vấn đều liên quan đến một loại tiêu chí lựa chọn. Trong trường hợp này, bạn phải xác định những toán hạng và các toán tử cần thiết trong tiêu chí của bạn. Đảm bảo kiểu dữ liệu và độ chi tiết của dữ liệu trong tiêu chí so sánh là chính xác.

a. So sánh đơn giản. Trong hầu hết các trường hợp, bạn sẽ so sánh một giá trị duy nhất - ví dụ, `P_PRICE > 10`.

b. Một giá trị đến các giá trị khác nhau. Nếu bạn đang so sánh một giá trị duy nhất với nhiều giá trị, bạn có thể cần sử dụng toán tử so sánh IN - ví dụ, `V_STATE IN ('FL', 'TN', 'GA')`.

c. Các so sánh lồng nhau. Trong những trường hợp khác, bạn có thể cần có một số tiêu chí lựa chọn lồng nhau liên quan đến các truy vấn phụ - ví dụ, `P_PRICE >= (SELECT AVG (P_PRICE) FROM PRODUCT)`.

d. Chọn dữ liệu theo nhóm. Trong các trường hợp khác, tiêu chí lựa chọn có thể áp dụng không phải cho dữ liệu nguyên thủy mà là cho dữ liệu tổng hợp. Trong những trường hợp đó, bạn cần sử dụng mệnh đề HAVING.

5. Xác định thứ tự hiển thị đầu ra. Cuối cùng, đầu ra yêu cầu có thể được sắp xếp theo một hoặc nhiều cột. Trong những trường hợp đó, bạn cần sử dụng mệnh đề ORDER BY. Hãy nhớ rằng mệnh đề ORDER BY là một trong những hoạt động chi phí tài nguyên cao nhất đối với DBMS.



## 11-7 DBMS Performance Tuning

Tinh chỉnh hiệu suất của DBMS bao gồm các nhiệm vụ toàn cầu như quản lý các quy trình DBMS trong bộ nhớ chính (phân bổ bộ nhớ cho mục đích lưu trữ tạm thời) và quản lý các cấu trúc trong bộ nhớ vật lý (phân bổ không gian cho các tệp dữ liệu).

Tinh chỉnh hiệu suất của DBMS cũng bao gồm áp dụng một số thực tiễn được kiểm tra trong phần trước đó. Ví dụ, DBA phải làm việc với các nhà phát triển để đảm bảo các truy vấn hoạt động như mong đợi - tạo chỉ mục để tăng tốc độ phản hồi truy vấn và tạo ra các thống kê cơ sở dữ liệu được yêu cầu bởi trình tối ưu hóa dựa trên chi phí.

Tinh chỉnh hiệu suất của DBMS tại điểm máy chủ tập trung vào cài đặt các thông số được sử dụng để:

- Bộ nhớ cache dữ liệu. Kích thước cache dữ liệu phải được đặt đủ lớn để cho phép phục vụ càng nhiều yêu cầu dữ liệu nhất có thể từ cache. Mỗi DBMS đều có các thiết lập điều khiển kích thước của cache dữ liệu; một số DBMS có thể yêu cầu khởi động lại. Cache này được chia sẻ giữa tất cả người dùng cơ sở dữ liệu. Hầu hết tài nguyên bộ nhớ chính sẽ được phân bổ cho cache dữ liệu.
- Bộ nhớ cache SQL. Bộ nhớ cache SQL lưu trữ các câu lệnh SQL được thực thi gần đây nhất (sau khi các câu lệnh SQL đã được phân tích bởi trình tối ưu hóa). Thông thường, nếu bạn có một ứng dụng với nhiều người dùng truy cập vào cơ sở dữ liệu, cùng một truy vấn sẽ được gửi bởi nhiều người dùng khác nhau. Trong những trường hợp đó, DBMS sẽ phân tích truy vấn chỉ một lần và thực hiện nhiều lần, sử dụng cùng kế hoạch truy cập. Bằng cách đó, yêu cầu SQL thứ hai và các yêu cầu SQL sau cho cùng một truy vấn sẽ được phục vụ từ cache SQL, bỏ qua giai đoạn phân tích.
- Bộ nhớ cache sắp xếp. Cache sắp xếp được sử dụng như một khu vực lưu trữ tạm thời cho các hoạt động ORDER BY hoặc GROUP BY, cũng như cho các chức năng tạo chỉ mục.
- Chế độ tối ưu trình. Hầu hết các DBMS hoạt động trong một trong hai chế độ tối ưu hóa: dựa trên chi phí hoặc dựa trên quy tắc. Các DBMS khác tự động xác định chế độ tối ưu hóa dựa trên sự có sẵn của thống kê cơ sở dữ liệu. Ví dụ, DBA chịu trách nhiệm tạo ra các thống kê cơ sở dữ liệu được sử dụng bởi trình tối ưu hóa dựa trên chi phí. Nếu các thống kê không có sẵn, DBMS sử dụng trình tối ưu hóa dựa trên quy tắc.

Từ góc độ hiệu suất, sẽ lý tưởng nếu toàn bộ cơ sở dữ liệu được lưu trữ trong bộ nhớ chính để giảm thiểu việc truy cập đĩa tốn kém. Đó là lý do tại sao một số nhà cung cấp cơ sở dữ liệu cung cấp tùy chọn cơ sở dữ liệu trong bộ nhớ cho các sản phẩm chính của họ. Các hệ thống cơ sở dữ liệu trong bộ nhớ được tối ưu hóa để lưu trữ một phần lớn (nếu không phải tất cả) của cơ sở dữ liệu trong bộ nhớ chính (RAM) thay vì lưu trữ thứ cấp (đĩa). Các hệ thống này đang trở nên phổ biến vì yêu cầu hiệu suất ngày càng cao của các ứng dụng cơ sở dữ liệu hiện đại (như phân tích kinh doanh và dữ liệu lớn), chi phí giảm và các tiến bộ về công nghệ của các thành phần (như flash-memory và ổ đĩa bán dẫn.) Mặc dù loại cơ sở dữ liệu này "loại bỏ" các chương ngại về truy cập đĩa, chúng vẫn phải tuân thủ các quy tắc tối ưu hóa truy vấn và hiệu suất, đặc biệt là khi đối mặt với cơ sở dữ liệu thiết kế kém hoặc các câu lệnh SQL viết kém.

Mặc dù các cơ sở dữ liệu trong bộ nhớ đang tạo ra một niềm tin trong các thị trường được lựa chọn, hầu hết các cài đặt cơ sở dữ liệu vẫn phụ thuộc vào dữ liệu được lưu trữ trên các ổ đĩa đĩa. Đó là lý do vì sao quản lý chi tiết lưu trữ vật lý của các tệp dữ liệu chơi vai trò quan trọng trong tinh chỉnh hiệu suất của DBMS. Lưu ý các khuyến nghị chung sau đối với lưu trữ vật lý của cơ sở dữ liệu:

- Sử dụng I / O tăng tốc. Loại thiết bị này sử dụng ổ đĩa bán dẫn flash (SSD) để lưu trữ cơ sở dữ liệu. Một SSD không có bất kỳ bộ phận di chuyển nào và, do đó, thực hiện các hoạt động I / O với tốc độ cao hơn so với các ổ đĩa quay truyền thống. I / O tăng tốc cung cấp tỷ lệ hiệu suất giao dịch cao và giảm độ cạnh tranh gây ra bởi các ổ đĩa lưu trữ điển hình.

- \* In-memory database : Một cơ sở dữ liệu được tối ưu hóa để lưu trữ một phần lớn (nếu không phải tất cả) của cơ sở dữ liệu trong bộ nhớ chính (RAM) thay vì lưu trữ thứ cấp (đĩa).

- \* I/O accelerator : Một thiết bị được sử dụng để cải thiện tốc độ xử lý cho hoạt động input/output.

- Sử dụng RAID (hệ thống đĩa độc lập dự phòng) để cung cấp cải thiện hiệu suất và độ bền, và một sự cân bằng giữa chúng. Độ bền có nghĩa là trong trường hợp xảy ra sự cố, dữ liệu có thể được tái tạo và khôi phục. Hệ thống RAID sử dụng nhiều đĩa để tạo ra các đĩa ảo (khối lưu trữ) được tạo thành từ một số đĩa cá nhân. Bảng 11.7 mô tả các cấu hình RAID phổ biến nhất.

TABLE 11.7

## COMMON RAID LEVELS

| RAID LEVEL | DESCRIPTION                                                                                                                                                                                                                                                                                                              |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0          | The data blocks are spread over separate drives. Also known as <i>striped array</i> . Provides increased performance but no fault tolerance. Requires a minimum of two drives.                                                                                                                                           |
| 1          | The same data blocks are written (duplicated) to separate drives. Also referred to as <i>mirroring</i> or <i>duplexing</i> . Provides increased read performance and fault tolerance via data redundancy. Requires a minimum of two drives.                                                                              |
| 3          | The data is striped across separate drives, and parity data is computed and stored in a dedicated drive. (Parity data is specially generated data that permits the reconstruction of corrupted or missing data.) Provides good read performance and fault tolerance via parity data. Requires a minimum of three drives. |
| 5          | The data and the parity data is striped across separate drives. Provides good read performance and fault tolerance via parity data. Requires a minimum of three drives.                                                                                                                                                  |
| 1+0        | The data blocks are spread over separate drives and mirrored (duplicated). This arrangement provides both speed and fault tolerance. This is the recommended RAID configuration for most database installations (if cost is not an issue).                                                                               |

• Giảm thiểu việc xung đột đĩa. Sử dụng nhiều khối lưu trữ độc lập với các trục độc lập (đĩa quay) để giảm thiểu các chu kỳ đĩa cứng. Hãy nhớ rằng cơ sở dữ liệu được tạo thành từ nhiều không gian bảng, mỗi không gian này có một chức năng cụ thể. Lần lượt, mỗi không gian bảng được tạo thành từ một số tệp dữ liệu trong đó dữ liệu được lưu trữ. Một cơ sở dữ liệu nên có ít nhất các không gian bảng sau:

- Không gian bảng hệ thống. Được sử dụng để lưu trữ bảng từ điển dữ liệu. Đây là không gian bảng được truy cập nhiều nhất và nên được lưu trữ trên một khối lưu trữ riêng.
- Không gian bảng dữ liệu người dùng. Được sử dụng để lưu trữ dữ liệu người dùng cuối. Bạn nên tạo ra nhiều không gian bảng dữ liệu người dùng và các tệp dữ liệu cần thiết để cân bằng hiệu suất và tính sử dụng. Ví dụ, bạn có thể tạo và gán một không gian bảng dữ liệu người dùng khác nhau cho mỗi ứng dụng và mỗi nhóm người dùng riêng biệt, nhưng điều này không cần thiết đối với mỗi người dùng.
- Không gian bảng chỉ mục. Được sử dụng để lưu trữ chỉ mục. Bạn có thể tạo và gán một không gian bảng chỉ mục khác nhau cho mỗi ứng dụng và mỗi nhóm người dùng. Tệp dữ liệu của không gian bảng chỉ mục nên được lưu trữ trên một khối lưu trữ khác với các tệp dữ liệu người dùng hoặc các tệp dữ liệu hệ thống.
- Không gian bảng tạm thời. Được sử dụng làm khu vực lưu trữ tạm thời cho các hoạt động trộn, sắp xếp hoặc tổng hợp tập hợp. Bạn có thể tạo và gán một không gian bảng tạm thời khác nhau cho mỗi ứng dụng và mỗi nhóm người dùng.
- Không gian bảng rollback segment. Được sử dụng cho mục đích khôi phục giao dịch.

- Đặt các bảng sử dụng cao vào không gian bảng riêng của chúng để cơ sở dữ liệu giảm thiểu xung đột với các bảng khác.

**RAID:** Một từ viết tắt cho Hệ thống đĩa độc lập dự phòng (Redundant Array of Independent Disks). Hệ thống RAID sử dụng nhiều đĩa để tạo ra các đĩa ảo (khối lưu trữ) được tạo thành từ một số đĩa cá nhân. Hệ thống RAID cung cấp cải thiện hiệu suất, độ bền và một sự cân bằng giữa hai yếu tố này.

- Gán các tệp dữ liệu riêng biệt trên các khối lưu trữ riêng biệt cho các chỉ mục, hệ thống và bảng sử dụng cao. Điều này đảm bảo các hoạt động chỉ mục sẽ không xung đột với dữ liệu người dùng cuối hoặc hoạt động truy cập vào bảng từ điển dữ liệu. Một lợi ích khác của phương pháp này là bạn có thể sử dụng kích thước khối đĩa khác nhau trên các khối lưu trữ khác nhau. Ví dụ: khối lưu trữ dữ liệu có thể sử dụng kích thước khối 16 K, trong khi khối lưu trữ chỉ mục có thể sử dụng kích thước khối 8 K. Hãy nhớ rằng kích thước bản ghi chỉ mục thường nhỏ hơn, và bằng cách thay đổi kích thước khối, bạn sẽ giảm thiểu các hoạt động xung đột và giảm thiểu hoạt động I/O. Điều này rất quan trọng; nhiều nhà quản trị cơ sở dữ liệu bỏ qua chỉ mục như một nguồn gây ra xung đột. Bằng cách sử dụng các khối lưu trữ riêng biệt và các kích thước khối khác nhau, các hoạt động I/O trên dữ liệu và chỉ mục sẽ xảy ra bất đồng bộ (vào các thời điểm khác nhau); quan trọng hơn, khả năng ghi chặn các hoạt động đọc được giảm xuống, vì khóa trang thường khóa ít bản ghi hơn.

- Tận dụng các tổ chức lưu trữ bảng khác nhau có sẵn trong cơ sở dữ liệu. Ví dụ, trong Oracle, xem xét việc sử dụng các bảng tổ chức chỉ mục (IOT); trong SQL Server, xem xét các bảng chỉ mục gom nhóm. **Một bảng tổ chức chỉ mục** (hoặc **bảng chỉ mục gom nhóm**) là một bảng lưu trữ dữ liệu người dùng cuối và dữ liệu chỉ mục ở các vị trí liên tiếp trên bộ nhớ lưu trữ vĩnh viễn. Loại tổ chức lưu trữ này cung cấp lợi thế về hiệu suất cho các bảng thường được truy cập thông qua một trình tự chỉ mục cụ thể, vì chỉ mục chứa cả khóa chỉ mục cũng như các hàng dữ liệu. Do đó, Hệ quản trị CSDL thường thực hiện ít hoạt động I/O hơn.

- Phân vùng bảng dựa trên việc sử dụng. Một số RDBMS hỗ trợ phân vùng ngang của bảng dựa trên thuộc tính. (Xem Chương 12, Hệ thống quản lý Cơ sở dữ liệu Phân tán.) Bằng cách làm như vậy, một yêu cầu SQL duy nhất có thể được xử lý bởi nhiều bộ xử lý dữ liệu. Đặt các phân vùng bảng gần với nơi chúng được sử dụng nhiều nhất.

- Sử dụng các bảng phi chuẩn hóa khi thích hợp. Nói cách khác, bạn có thể cải thiện hiệu suất bằng cách chuyển bảng từ dạng chuẩn hóa cao hơn thành dạng chuẩn hóa thấp hơn - thường là từ dạng chuẩn hóa thứ ba thành dạng chuẩn hóa thứ hai. Kỹ thuật này thêm sự trùng lặp dữ liệu, nhưng giảm thiểu các hoạt động kết nối. (Phi chuẩn hóa đã được thảo luận trong Chương 6, Chuẩn hóa các bảng Cơ sở dữ liệu.(Normalization of Database Tables.))

- Lưu trữ các thuộc tính đã tính toán và tổng hợp trong bảng. Tóm lại, hãy sử dụng các thuộc tính tạo ra trong các bảng của bạn. Ví dụ, bạn có thể thêm tổng số hóa đơn, số lượng thuế và tổng cộng trong bảng INVOICE. Việc sử dụng các thuộc tính tạo ra giảm thiểu các phép tính trong các truy vấn và hoạt động kết nối, đặc biệt là trong quá trình thực thi các truy vấn tổng hợp.

### 11-8 Query Optimization Example

Sau khi đã học cơ bản về tối ưu hóa truy vấn, bạn đã sẵn sàng thử nghiệm kiến thức mới của mình. Một ví dụ đơn giản minh họa cách trình tối ưu hóa truy vấn hoạt động và cách bạn có thể giúp cho nó hoạt động tốt hơn. Ví dụ này dựa trên các bảng QOVENDOR và QOPRODUCT, tương tự như các bảng bạn đã sử dụng trong các chương trước đó. Tuy nhiên, tiền tố QO được sử dụng cho tên bảng để đảm bảo rằng bạn không ghi đè lên các bảng trước đó.

Để thực hiện ví dụ tối ưu hóa truy vấn này, bạn sẽ sử dụng giao diện Oracle SQL\*Plus. Một số công việc tiền xử lý phải được thực hiện trước khi bạn có thể bắt đầu thử nghiệm tối ưu hóa truy vấn, được giải thích trong các bước sau đây:

1. Đăng nhập vào Oracle SQL\*Plus bằng tên người dùng và mật khẩu được cung cấp bởi giảng viên của bạn.

2. Tạo một bộ các bảng mới, bằng cách sử dụng tệp lệnh QRYOPTDATA.SQL (có sẵn tại đường dẫn [www.cengagebrain.com](http://www.cengagebrain.com)). Bước này là cần thiết để Oracle có một bộ các bảng mới và các bảng mới này không chứa bất kỳ thống kê nào. Tại dấu nhắc SQL>, gõ:

```
@path\QRYOPTDATA.SQL
```

ở đó path là vị trí của tệp trên máy tính của bạn.

3. Tạo PLAN\_TABLE, đây là một bảng đặc biệt được sử dụng bởi Oracle để lưu trữ thông tin kế hoạch truy cập cho một truy vấn cụ thể. Người dùng cuối sau đó có thể

truy vấn PLAN\_TABLE để xem Oracle sẽ thực thi truy vấn như thế nào. Để tạo PLAN\_TABLE, chạy tệp lệnh UTLXPLAN.SQL trong thư mục RDBMS\ADMIN của cài đặt Oracle RDBMS của bạn. Tệp lệnh UTLXPLAN.SQL cũng có sẵn tại [www.cengagebrain.com](http://www.cengagebrain.com). Tại dấu nhắc SQL>, gõ:

```
@path\UTLXPLAN.SQL
```

Bạn sử dụng lệnh EXPLAIN PLAN để lưu trữ kế hoạch thực thi của một truy vấn SQL trong PLAN\_TABLE. Sau đó, bạn sử dụng lệnh SELECT \* FROM TABLE(DBMS\_XPLAN.

DISPLAY) để hiển thị lịch truy cập cho một câu lệnh SQL cụ thể.

**Note:** Oracle, MySQL và SQL Server đều mặc định sử dụng tối ưu hóa dựa trên chi phí. Trong Oracle, nếu không có thống kê bảng có sẵn, hệ thống quản lý cơ sở dữ liệu (DBMS) sẽ trở về tối ưu hóa dựa trên các quy tắc (rule-based optimizer).

Để xem lịch truy cập được sử dụng bởi DBMS để thực hiện truy vấn của bạn, sử dụng lệnh EXPLAIN PLAN và SELECT, như được hiển thị trong Hình 11.5. Lưu ý rằng câu lệnh SQL đầu tiên tạo ra thống kê cho bảng QOVENDOR. Ngoài ra, kế hoạch truy cập ban đầu trong Hình 11.5 sử dụng quét toàn bộ bảng trên bảng QOVENDOR, và chi phí của kế hoạch là 4.

FIGURE 11.5 INITIAL EXPLAIN PLAN

```

SQL> ANALYZE TABLE QOVENDOR COMPUTE STATISTICS;
Table analyzed.
SQL> EXPLAIN PLAN FOR SELECT * FROM QOVENDOR WHERE U_NAME LIKE 'B%' ORDER BY U_AREACODE;
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 1808591659

+-----+-----+-----+-----+-----+-----+-----+
| Id | Operation | Name | Rows | Bytes | Cost | CPU | Time |
+-----+-----+-----+-----+-----+-----+-----+
| 0 | SELECT STATEMENT | | | | | | |
| 1 | SORT ORDER BY | | | | | | |
| 2 | TABLE ACCESS FULL | QOVENDOR | | | | | |
+-----+-----+-----+-----+-----+-----+-----+

Predicate Information (identified by operation id):
PLAN_TABLE_OUTPUT

2 - filter("U_NAME" LIKE 'B%')
14 rows selected.
SQL>
  
```

Bây giờ hãy tạo một chỉ mục trên V\_AREACODE (lưu ý rằng V\_AREACODE được sử dụng trong mệnh đề ORDER BY) và xem nó làm thay đổi kế hoạch truy cập được tạo ra bởi bộ tối ưu hóa dựa trên chi phí. Kết quả được hiển thị trong Hình 11.6.

FIGURE 11.6 EXPLAIN PLAN AFTER INDEX ON V\_AREACODE

```

SQL> CREATE INDEX QOV_NDX1 ON QOVENDOR(V_AREACODE);
Index created.
SQL> ANALYZE TABLE QOVENDOR COMPUTE STATISTICS;
Table analyzed.
SQL> EXPLAIN PLAN FOR SELECT * FROM QOVENDOR WHERE U_NAME LIKE 'B%' ORDER BY U_AREACODE;
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 641227332

   Id  Operation                   Name              Rows  Bytes  Cost (%CPU)  Time     1
  ---  -
0     SELECT STATEMENT                  1      38       2   (0)    00:00:01
1     TABLE ACCESS BY INDEX ROWID    QOVENDOR          1      38       2   (0)    00:00:01
2     INDEX FULL SCAN                  QOV_NDX1          15           1   (0)    00:00:01

Predicate Information (identified by operation id):
PLAN_TABLE_OUTPUT

1 - filter('U_NAME' LIKE 'B%')
14 rows selected.
SQL>

```

Trong Hình 11.6, lưu ý rằng kế hoạch truy cập mới giảm chi phí thực thi truy vấn đi 30 phần trăm! Cũng lưu ý rằng kế hoạch mới quét chỉ mục QOV\_NDX1 và truy cập các hàng QOVENDOR, sử dụng ID hàng chỉ mục. (Hãy nhớ rằng truy cập bằng ID hàng là một trong những phương pháp truy cập nhanh nhất.) Trong trường hợp này, việc tạo chỉ mục QOV\_NDX1 đã có tác động tích cực đến kết quả tối ưu hóa truy vấn chung.

Đôi khi, chỉ mục không nhất thiết giúp tối ưu hóa truy vấn, chẳng hạn khi bạn có chỉ mục trên các bảng nhỏ hoặc khi truy vấn truy cập một phần trăm lớn hàng trong bảng. Lưu ý xem điều gì xảy ra khi bạn tạo một chỉ mục trên V\_NAME. Kế hoạch truy cập mới được hiển thị trong Hình 11.7. (Lưu ý rằng V\_NAME được sử dụng trong mệnh đề WHERE như một toán hạng biểu thức điều kiện.)

Như bạn có thể thấy trong Hình 11.7, việc tạo chỉ mục thứ hai không giúp tối ưu hóa truy vấn. Tuy nhiên, vào một số dịp, bộ tối ưu hóa có thể sử dụng một chỉ mục, nhưng



không được thực thi vì cách truy vấn được viết. Ví dụ, Hình 11.8 cho thấy kế hoạch truy cập cho một truy vấn khác sử dụng cột V\_NAME.

FIGURE 11.7 EXPLAIN PLAN AFTER INDEX ON V\_NAME

```

SQL> CREATE INDEX QOV_NDX2 ON QOVENDOR(U_NAME);
Index created.
SQL> ANALYZE TABLE QOVENDOR COMPUTE STATISTICS;
Table analyzed.
SQL> EXPLAIN PLAN FOR SELECT * FROM QOVENDOR WHERE U_NAME LIKE 'B%' ORDER BY U_AREACODE;
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 641227332

   Id Operation                Name           Rows  Bytes Cost (<CPU>) Time
--  -
0 SELECT STATEMENT              1      38    2 (<0>) 00:00:01
1  TABLE ACCESS BY INDEX ROWID QOVENDOR             1      38    2 (<0>) 00:00:01
2    INDEX FULL SCAN            QOV_NDX1       15         1    1 (<0>) 00:00:01

Predicate Information (identified by operation id):
PLAN_TABLE_OUTPUT

1 - filter("U_NAME" LIKE 'B%')
14 rows selected.
SQL>

```

FIGURE 11.8 ACCESS PLAN USING INDEX ON V\_NAME

```

SQL> EXPLAIN PLAN FOR SELECT U_NAME, P_CODE FROM QOVENDOR U, QOPRODUCT P
2 WHERE U.U_CODE = P.U_CODE AND U_NAME = 'ORDVA, Inc.';
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 4146133223

   Id Operation                                Name           Rows  Bytes Cost (<CPU>) Time
--  -
0 SELECT STATEMENT                          2      62    5 (<0>) 00:00:01
1  HASH JOIN                                2      62    5 (<0>) 00:00:01
2    TABLE ACCESS BY INDEX ROWID BATCHED QOVENDOR             1      17    2 (<0>) 00:00:01
3      INDEX RANGE SCAN                QOV_NDX2             1         1    1 (<0>) 00:00:01
4    TABLE ACCESS FULL                QOPRODUCT          14     196    3 (<0>) 00:00:01

PLAN_TABLE_OUTPUT

Predicate Information (identified by operation id):
1 - access("U"."U_CODE"="P"."U_CODE")
3 - access("U_NAME"='ORDVA, Inc.')
4 - filter("P"."U_CODE" IS NOT NULL)
18 rows selected.
SQL>

```

Trong Hình 11.8, lưu ý rằng kế hoạch truy cập cho truy vấn mới này sử dụng chỉ mục QOV\_NDX2 trên cột V\_NAME. Điều gì sẽ xảy ra nếu bạn viết cùng một truy vấn, sử dụng hàm UPPER trên V\_NAME? Kết quả được minh họa trong Hình 11.9.

FIGURE 11.9 ACCESS PLAN USING FUNCTIONS ON INDEXED COLUMNS

```

SQL> EXPLAIN PLAN FOR SELECT U_NAME, P_CODE FROM QOUENDOR U, QOPRODUCT P
2 WHERE U.U_CODE = P.U_CODE AND UPPER(U_NAME) = 'ORDUA, INC.';

Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 1347998970

+----+-----+-----+-----+-----+-----+-----+-----+
| Id | Operation          | Name                | Rows  | Bytes | Cost | (%CPU)| Time     |
+----+-----+-----+-----+-----+-----+-----+-----+
| 0  | SELECT STATEMENT   |                     |      1 |    31 |     5 |    (0)| 00:00:01 |
|* 1  |  HASH JOIN         |                     |      1 |    31 |     5 |    (0)| 00:00:01 |
| 2  |    VIEW            | index$_join$_001    |      1 |    17 |     2 |    (0)| 00:00:01 |
|* 3  |      HASH JOIN     |                     |      1 |    17 |     1 |    (0)| 00:00:01 |
|* 4  |        INDEX FAST FULL SCAN | QOU_NDX2            |      1 |    17 |     1 |    (0)| 00:00:01 |
| 5  |          INDEX FAST FULL SCAN | SYS_C0058569        |      1 |    17 |     1 |    (0)| 00:00:01 |
+----+-----+-----+-----+-----+-----+-----+-----+
PLAN_TABLE_OUTPUT

+----+-----+-----+-----+-----+-----+-----+-----+
|* 6  |  TABLE ACCESS FULL | QOPRODUCT            |     14 |    196 |     3 |    (0)| 00:00:01 |
+----+-----+-----+-----+-----+-----+-----+-----+

Predicate Information (identified by operation id):

   1 - access("U"."U_CODE"="P"."U_CODE")
   3 - access(ROWID=ROWID)
   4 - filter(UPPER("U_NAME")='ORDUA, INC.')
   6 - filter("P"."U_CODE" IS NOT NULL)

21 rows selected.

SQL>

```

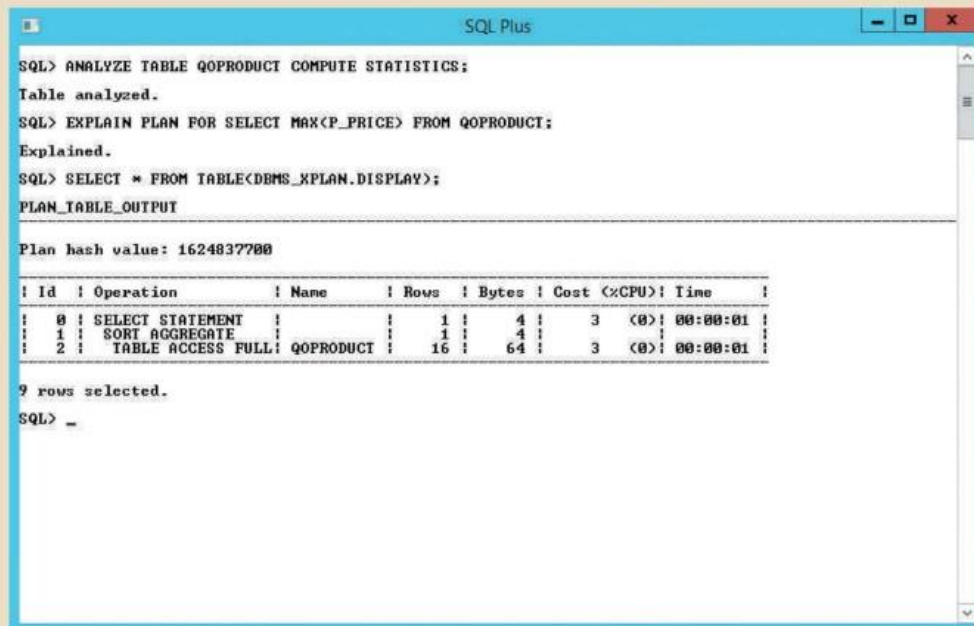
Như Hình 11.9 cho thấy, việc sử dụng hàm trên một cột có chỉ mục đã khiến cho Hệ quản trị cơ sở dữ liệu thực hiện các hoạt động bổ sung có thể tăng chi phí của truy vấn. Cùng một truy vấn có thể cho ra các chi phí khác nhau nếu các bảng của bạn chứa nhiều hàng hơn và nếu độ dày của chỉ mục khác nhau.

Bây giờ sử dụng bảng QOPRODUCT để minh họa cách chỉ mục có thể giúp đỡ khi chạy các truy vấn hàm tổng hợp. Ví dụ, Hình 11.10 cho thấy kế hoạch truy cập cho một câu lệnh SELECT sử dụng hàm tổng hợp MAX(P\_PRICE). Kế hoạch này sử dụng toàn bộ bảng quét với tổng chi phí là 3.

Chi phí là 3 đã rất thấp, nhưng bạn có thể cải thiện hiệu suất truy vấn trước đó bằng cách tạo một chỉ mục trên cột P\_PRICE. Hình 11.11 cho thấy cách giảm chi phí kế hoạch đi hai phần ba sau khi tạo chỉ mục và phân tích bảng QOPRODUCT. Hãy lưu ý rằng phiên bản thứ hai của kế hoạch truy cập chỉ sử dụng chỉ mục QOP\_NDX2 để trả

lời truy vấn; bảng QOPRODUCT không bao giờ được truy cập.

FIGURE 11.10 FIRST EXPLAIN PLAN: AGGREGATE FUNCTION ON A NON-INDEXED COLUMN



```

SQL> ANALYZE TABLE QOPRODUCT COMPUTE STATISTICS;
Table analyzed.
SQL> EXPLAIN PLAN FOR SELECT MAX(P_PRICE) FROM QOPRODUCT;
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 1624837700

   Id  Operation          Name              Rows  Bytes  Cost (%CPU)  Time
--  -  -
   0   SELECT STATEMENT                1      4       3   (0)   00:00:01
   1   SORT AGGREGATE                  1      4       4   (0)   00:00:01
   2   TABLE ACCESS FULL QOPRODUCT    16     64       3   (0)   00:00:01

9 rows selected.

SQL> _

```

FIGURE 11.11 SECOND EXPLAIN PLAN: AGGREGATE FUNCTION ON AN INDEXED COLUMN



```

SQL> CREATE INDEX QOP_NDX2 ON QOPRODUCT(P_PRICE);
Index created.
SQL> ANALYZE TABLE QOPRODUCT COMPUTE STATISTICS;
Table analyzed.
SQL> EXPLAIN PLAN FOR SELECT MAX(P_PRICE) FROM QOPRODUCT;
Explained.
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
PLAN_TABLE_OUTPUT

Plan hash value: 3880272194

   Id  Operation          Name              Rows  Bytes  Cost (%CPU)  Time
--  -  -
   0   SELECT STATEMENT                1      4       1   (0)   00:00:01
   1   SORT AGGREGATE                  1      4       1   (0)   00:00:01
   2   INDEX FULL SCAN (MIN/MAX) QOP_NDX2 1      4       1   (0)   00:00:01

9 rows selected.

SQL> _

```

Mặc dù vài ví dụ trong phần này cho thấy sự quan trọng của việc chọn chỉ mục đúng cho tối ưu hóa truy vấn, bạn cũng đã thấy các ví dụ trong đó việc tạo chỉ mục không cải thiện hiệu suất truy vấn. Là một DBA, bạn nên nhận thức rằng mục tiêu chính là tối ưu hóa hiệu suất cơ sở dữ liệu tổng thể - không chỉ cho một truy vấn đơn lẻ mà cho tất cả các yêu cầu và loại truy vấn. Hầu hết các hệ thống cơ sở dữ liệu cung cấp các

công cụ đồ họa tiên tiến để theo dõi và kiểm tra hiệu suất. Ví dụ, Hình 11.12, 11.13 và 11.14 cho thấy biểu đồ đại diện cho kế hoạch truy cập sử dụng các công cụ Oracle, MySQL và Microsoft SQL Server.

FIGURE 11.12 ORACLE TOOLS FOR QUERY OPTIMIZATION

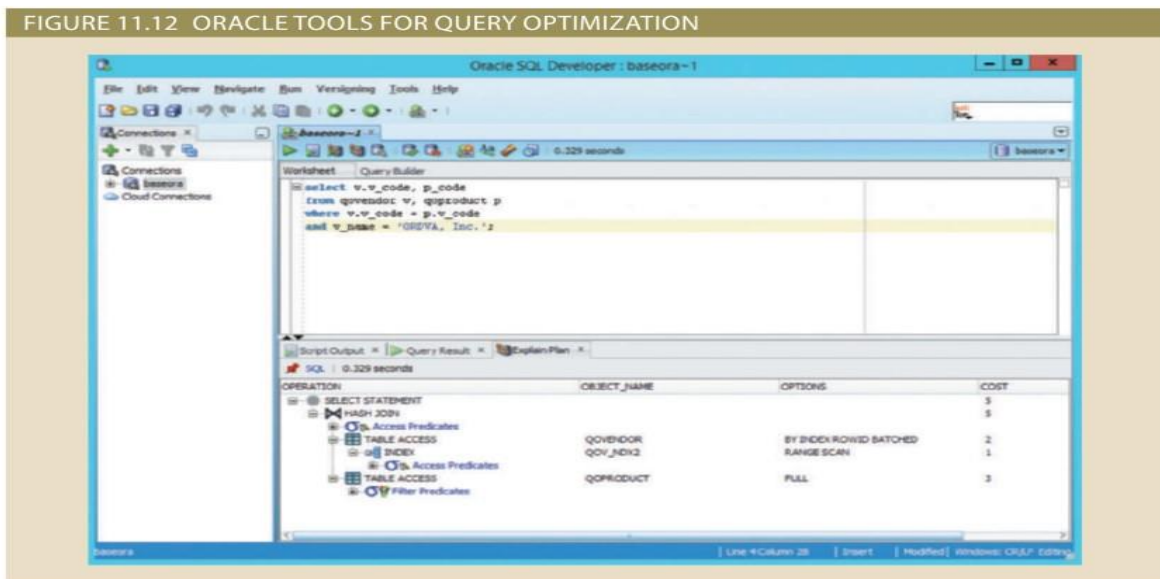


FIGURE 11.13 MYSQL TOOLS FOR QUERY OPTIMIZATION

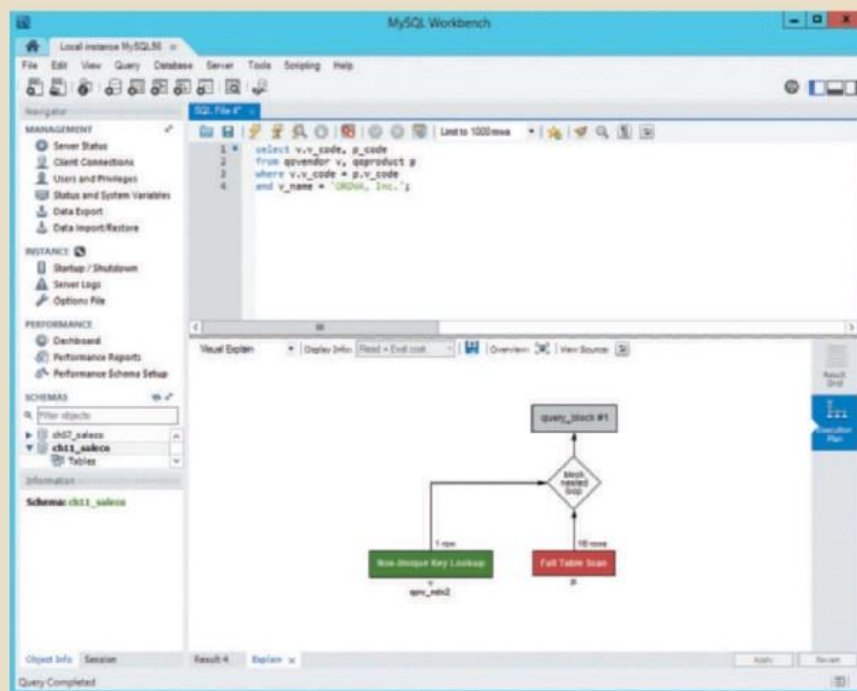
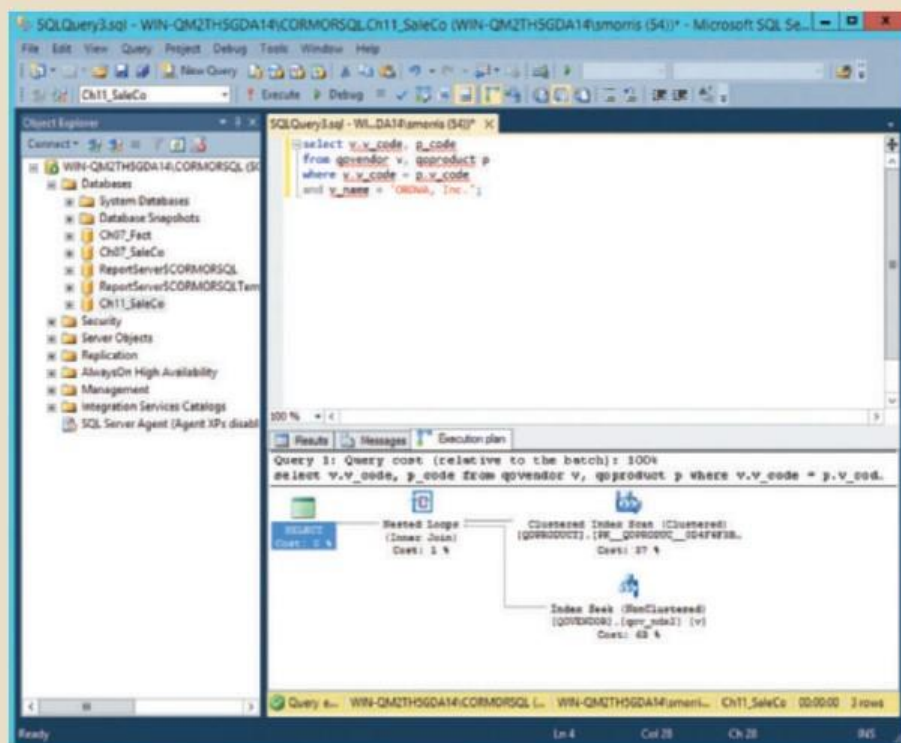


FIGURE 11.14 MICROSOFT SQL SERVER TOOLS FOR QUERY OPTIMIZATION



Summary

- Tối ưu hiệu suất cơ sở dữ liệu đề cập đến một tập hợp các hoạt động và thủ tục được thiết kế để đảm bảo một truy vấn người dùng được xử lý bởi DBMS trong khoảng thời gian ngắn nhất.
- Tối ưu hiệu suất SQL đề cập đến các hoạt động trên phía khách hàng được thiết kế để tạo mã SQL trả về câu trả lời chính xác trong thời gian ngắn nhất, sử dụng tối thiểu tài nguyên tại phía máy chủ.
- Tối ưu hiệu suất DBMS liên quan đến các hoạt động trên phía máy chủ để đảm bảo DBMS được cấu hình đúng để phản hồi các yêu cầu từ khách hàng nhanh nhất có thể trong khi tối ưu hóa tài nguyên hiện có.
- Thống kê cơ sở dữ liệu đề cập đến một số đo lường được thu thập bởi DBMS mô tả một bức tranh về các đặc điểm của các đối tượng cơ sở dữ liệu. DBMS thu thập thống kê về các đối tượng như bảng, chỉ mục và các tài nguyên khả dụng.
- DBMS xử lý truy vấn trong ba giai đoạn. Trong giai đoạn phân tích cú pháp, DBMS dùng cú pháp SQL để chọn kế hoạch truy cập/ thực thi hiệu quả nhất. Trong giai đoạn thực thi, DBMS thực thi câu truy vấn SQL bằng cách sử dụng kế hoạch thực hiện đã được chọn. Trong giai đoạn lấy dữ liệu, DBMS lấy dữ liệu và gửi bộ kết quả trở lại cho khách hàng.
- Chỉ mục rất quan trọng trong quá trình tăng tốc truy cập dữ liệu. Chỉ mục hỗ trợ tìm kiếm, sắp xếp và sử dụng các hàm tổng hợp và toán tử nối. Cải tiến tốc độ truy cập dữ liệu xảy ra bởi vì chỉ mục là một tập hợp được sắp xếp các giá trị chứa khóa chỉ mục và các con trỏ.
- Trong quá trình tối ưu truy vấn, DBMS phải chọn chọn chỉ mục sử dụng nào, cách thực hiện các toán tử nối, bảng nào được sử dụng trước và như vậy. Mỗi DBMS có thuật toán riêng để xác định cách tiếp cận tốt nhất cho việc truy xuất dữ liệu.
- SQL tối ưu hiệu suất xử lý câu truy vấn và tối ưu hiệu suất cơ sở dữ liệu bao gồm các nhiệm vụ như quản lý quá trình DBMS trong bộ nhớ chính (phân bổ bộ nhớ cho mục đích caching) và quản lý các cấu trúc trong bộ nhớ vật lý (phân bổ không gian cho các tệp dữ liệu).